

大 作 业

揭棋对弈程序设计

最终报告（初版）

小组成员

张恒基（组长）	2024211301 / 2024210926
秦博宇	2024211302 / 2024210940
陈艺博	2024211302 / 2024210931
陈雨飞	2024211005

项目代号：Unveil

2026 年 6 月 18 日

北京邮电大学 • 计算机学院

目录

1. 项目总览	1
1.1. 项目背景	1
1.2. 六大目标	1
1.3. 项目规模	1
1.4. 技术栈	2
1.5. 项目结构	2
1.6. 验收主线	3
2. 总体架构	4
2.1. 模块依赖	4
2.2. 模块职责	4
2.3. 对局主流程	6
2.4. 核心类图	8
2.5. 坐标系统	9
2.6. 虚拟类型机制	9
2.7. 明子强化规则	10
2.8. 一次走子的完整调用链	10
3. WebSocket + JSON 通信协议规范	11
3.1. 基础约定	11
3.1.1. 术语定义	11
3.1.2. 技术约定	11
3.2. 棋子类型编码	12
3.2.1. 老师 JSON piece 枚举映射	12
3.2.2. 颜色编码	12
3.3. Move 对象规范	13
3.3.1. 字段规则	13
3.3.2. 翻子随机性机制	13
3.3.3. JSON move 对象格式	13
3.4. 完整消息类型规范	14
3.4.1. 客户端 → 服务器消息	14
3.4.2. 服务器 → 客户端消息	15
3.4.3. 公共数据结构	17
3.4.4. 游戏结果原因 (gameOver.reason)	18
3.4.5. 错误码	18
3.5. 典型通信时序	19
3.5.1. 匹配与先手协商	19
3.5.2. 正常走子与翻子	21

3.5.3. 非法着法	21
3.5.4. 超时判负	21
3.5.5. 认输	22
3.5.6. 提和流程	22
3.5.7. 复盘请求	23
3.5.8. 断线判负	24
3.6. 本组扩展消息总览	24
3.7. 协议实现状态	24
4. 规则引擎设计	26
4.1. 七种棋子走法速查	26
4.2. 暗子特殊规则	27
4.3. 暗子真实类型与虚拟类型	28
4.4. 服务端权威校验	28
4.5. 将军、送将与将帅照面	29
4.6. 校验流程	29
4.7. 规则引擎设计特点	30
4.8. 终局判定	30
4.9. 测试覆盖	31
5. AI 算法设计	32
5.1. 问题形式化	32
5.2. 规则引擎算法 (RuleValidator + Board)	32
5.2.1. 两层合法性校验	32
5.2.2. 暗子走法依据: getMoveType()	33
5.2.3. 揭棋特有规则 (算法分支)	33
5.3. 终局判定算法 (EndgameJudge)	33
5.4. AI 三档架构	34
5.4.1. Easy: 启发式 + 随机	34
5.4.2. Medium: Agent 编排 + Alpha-Beta	34
5.5. Alpha-Beta 搜索引擎 (OptimizedAlphaBeta)	35
5.5.1. 框架: Negamax + 迭代加深 (ID)	35
5.5.2. 剪枝与加速技术	35
5.5.3. 静态交换评估 SEE (StaticExchangeEvaluator)	36
5.5.4. Zobrist 哈希 (ZobristHash)	36
5.6. 局面评估函数 (EnhancedEvaluator)	36
5.6.1. 七维特征	36
5.6.2. 暗子子力处理 (Expectimax 思想的静态近似)	36
5.6.3. 概率偏置 (ProbabilityAgent)	37
5.7. Hard 档: Belief Sampling (BeliefAlphaBetaBot)	37
5.7.1. 算法流程	37

5.7.2. 剩余子力池约束 (BoardSampler)	37
5.7.3. 时间分配	37
5.7.4. 理论定位	37
5.8. 算法整体数据流	37
5.9. AI 测试	38
5.10. 已知算法局限	39
5.11. 三档 AI 代码对应表	39
5.12. Hard AI 选着流程	40
6. 客户端设计	40
6.1. 控制台客户端 (jieqi-client)	40
6.2. Web 前端 (jieqi-web)	41
6.2.1. 前端状态流	41
6.2.2. Web 界面展示	42
6.2.2.1. 登录与大厅	42
6.2.2.2. AI 对弈模式	44
6.2.2.3. 真人对战界面	46
6.2.2.4. 局内操作与聊天	46
6.2.2.5. 聊天室协议与实现	48
6.2.2.6. 终局结果	49
6.2.3. 复盘模式架构	50
6.3. Web 端当前进展与已知限制	50
6.3.1. 已完成 (可演示)	50
6.3.2. 待强化 / 已知限制	51
7. 棋谱与复盘	52
7.1. 三类产物	52
7.2. 帧编号模型	52
7.3. 产生时机	52
7.4. 复盘协议	53
7.5. 权限与视角	53
7.6. 前端复盘引擎	53
7.6.1. 核心数据模型	53
7.6.2. 完整流程	54
7.6.3. 服务端支持	54
7.6.4. 架构总结	54
7.6.5. 设计要点	54
8. 测试与质量	55
8.1. 测试汇总	55
8.2. 分模块结果	56
8.3. 关键测试场景 (摘录)	56

8.4. AI 性能观测	57
8.5. 需求与测试映射矩阵	57
8.6. 详细测试清单	58
8.6.1. jieqi-core 测试清单 (15 类)	58
8.7. jieqi-server 测试清单 (12 类)	59
8.8. jieqi-ai 测试清单 (5 类)	60
8.9. AI 性能基准	60
8.10. 测试框架与 CI	60
8.11. 测试质量评估	61
8.12. 已知未修	62
9. 部署与运行	62
9.1. 部署拓扑	62
9.2. 环境要求	63
9.2.1. 必装组件	63
9.2.2. 可选组件	64
9.2.3. JDK 环境核对清单 (答辩机必做)	64
9.3. 自检	64
9.3.1. verify.ps1 三步	64
9.4. 启动服务端	64
9.4.1. 方式对比 (推荐顺序)	65
9.4.2. 启动成功标志	65
9.4.3. 交互菜单 (jieqi-app Main 1-10)	66
9.5. 启动客户端	66
9.5.1. 控制台 WS 客户端	66
9.5.2. Web 前端	66
9.5.3. AI 经 WS (验收补充)	66
9.6. 完整演示流程	66
9.7. 一键演示	67
9.8. Docker (实验性)	67
9.8.1. 前置与启动	67
9.8.2. Dockerfile 两阶段	67
9.8.3. 端口与限制	68
9.8.4. Docker vs 本地	68
9.9. 脚本工具箱	68
9.10. 常见问题速查	68
9.10.1. 诊断命令	69
9.11. 启动成功标志	70
10. 产品与用户	70
10.1. 产品定位	70

10.2. 用户角色与需求	71
10.3. 用户旅程	72
10.3.1. 三条典型路径差异	73
10.3.2. 控制台旅程（验收补充）	73
10.4. 用户痛点与产品价值	74
10.5. 核心体验设计	74
10.5.1. 轻量身份设计	74
10.5.2. 模式分层设计	74
10.5.3. 对弈交互设计	75
10.5.4. 终局与复盘设计	75
10.6. 竞品选择依据	75
10.7. 竞品对比	75
10.8. 差异化价值	77
10.9. 功能完成度总览	78
10.9.1. 总览统计	78
10.9.2. 按模块统计	78
10.10. 产品不足与后续规划	79
11. 项目管理	79
11.1. 团队分工	80
11.2. Git 提交规范	82
11.3. 四大任务完成状态	83
11.3.1. 四大任务交叉验收	84
12. 演示与答辩	84
12.1. 演示时间线（6 - 8 分钟）	84
12.2. 关键命令备忘	85
12.3. 风险预案	86
12.4. 答辩高频问答	86
13. 总结与展望	86
13.1. 项目成果总览	87
13.2. 技术亮点总结	87
13.3. 已完成功能	88
13.4. 已知限制	89
13.4.1. P1 — 短期应强化	89
13.4.2. P2 — 中期可优化	89
13.4.3. P3 — 已知不追求（课设边界）	90
13.5. 后续规划	90
13.5.1. 版本路线图	90
13.5.2. V1.1（短期）— 规则与复盘加固	91
13.5.3. V2.0（中期）— 产品化与互操作	91

13.5.4. V3.0（远期）— 竞技与智能	92
13.5.5. 四大任务与版本对应	92
13.6. 关键子系统关联总览	92
13.7. 结语	93
14. 附录	93
14.1. A. TCP 文本帧扩展协议（摘要）	93
14.2. B. 文档体系完整清单	94
14.3. C. 术语速查	94
14.4. D. 构建命令速查	95
14.5. E. 版本历史	96

整合说明：第 3 章 = 完整 WebSocket JSON 协议规范（原 INTERFACE.typ v3.1）；第 4 - 11 章 = 设计/测试/部署/产品/答辩内容；附录 = TCP 扩展 + 术语表 + 命令速查。

1. 项目总览

1.1. 项目背景

揭棋是中国象棋变体：开局仅将/帅明置，其余 15 子暗置并按所在位置的原始角色走子；首次移动或吃子后随机翻开，明子按真实身份行棋。规则涉及暗子/明子差异、强化士象（明士可出九宫、明象可过河）、禁送将、将帅照面、长将长捉等复杂约束。本项目是北京邮电大学「揭棋对弈程序设计」课程大作业，代号 Unveil，由第一组（张恒基组长）完成。

1.2. 六大目标

No.	目标	状态
1	揭棋规则引擎 — 服务端权威校验，覆盖七种棋子走法、暗子约束、终局判定全链路	已实现
2	WebSocket + JSON 网络对弈 — 课程公共接口（端口 8887），匹配/房间/超时/聊天/提和/认输	已实现
3	三档 AI 博弈 — Easy 启发式 / Medium Alpha-Beta / Hard Belief Sampling 非完全信息搜索	已实现
4	棋谱与复盘 — 文字棋谱落盘 + 内存复盘时间线 + replay.json 持久化 + replayRequest/replayFrame 协议	已实现
5	工程化交付 — Maven 5 模块 + Fat JAR + verify.ps1 自检 + demo.ps1 演示	已实现
6	文档体系 — 34 份 Typst → 33 份 PDF，覆盖需求→设计→协议→部署→测试→产品→答辩全生命周期	已实现

1.3. 项目规模

指标	数值	说明
Maven 模块	5	jieqi-core / jieqi-server / jieqi-client / jieqi-ai / jieqi-app
Java 主代码	63 文件	约 8,500 行（含测试约 10,500）
自动化测试	142/142 通过	core 89 + ai 16 + server 37, 0 失败 0 跳过

Typst 文档	34 份	编译 33 份 PDF，覆盖 8 个类别
Web 前端	Vue 3 + Vite + Pinia	对局界面 + 终局查看 + 复盘控件

1.4. 技术栈

Java 21 • Maven 3.9+ • Java-WebSocket • Gson • JUnit 5 • Typst 0.14.2
(协议 PDF) • Docker Compose • Vue 3 + Vite + Pinia (Web 前端)

1.5. 项目结构

```

Unveil/
├─ pom.xml                # Maven 父 POM (Java 21 多模块)
├─ jieqi-core/            # 规则核心: Board, ChessPiece, RuleValidator,
Game,                      #   Protocol.java, json/, GameRecord,
├─ MoveNotation           #   WS(8887)/TCP(8888) 服务: WsGameServer,
├─ jieqi-server/          #   Room, Matchmaking, GameRecordStore
GameServer,               #   控制台客户端: GameClient, WsGameClient,
├─ jieqi-client/          #   搜索与评估: AiConfig, EasyRuleBot,
ConsoleUI                 #   BeliefAlphaBetaBot, AgentOrchestrator,
├─ jieqi-ai/              #   AlphaBetaSearch, TranspositionTable,
AlphaBetaBot,            #
├─ BoardSampler           # 启动入口: UnveilApp (1-10 菜单), Fat JAR 打
├─ jieqi-app/             #
包                          # Vue 3 前端: LoginView, LobbyView, GameView,
├─ jieqi-web/             #   ChessBoard.vue, game.ts(Pinia), ws.ts,
├─ chessRules.ts          # verify.ps1, demo.ps1, run-app.ps1, dev-
├─ scripts/               #   compile-docs.ps1, count-loc.ps1
server.ps1,              # Typst 文档体系: template.typ, INTERFACE.typ
├─ docs/                  #
v3.1,                    #   00-overview/ ~ 07-presentation/ (8 类 34
└─                         份)

```

Listing 1: 项目目录结构 (完整包路径见 §2 模块职责表)

1.6. 验收主线

本项目围绕五条课程验收主线展开。

验收主线	实现方式
规则正确性	所有正式对局走子由服务端 <code>Game.processMove</code> 统一进入 <code>RuleValidator</code> 与 <code>EndgameJudge</code> ；客户端仅承担交互提示，不作为权威规则源
网络互操作	以 WebSocket + JSON 为主协议（端口 8887），覆盖登录、匹配、房间、准备、先手协商、走子、聊天、提和、认输、超时、终局、复盘和重赛；控制台客户端与 Web 前端共用同一协议
AI 可解释性	三档 AI 分别对应 Easy（启发式排序 + 随机选择）、Medium（Agent 编排 + Alpha-Beta 搜索）、Hard（Belief Sampling 非完全信息搜索）
棋局可追溯	同时保存文字棋谱（ <code>records/*.jieqi</code> ）和 <code>ReplayFrame</code> 快照时间线；文字棋谱用于人工阅读与导出，快照用于逐步还原每一步后的棋盘状态，避免暗子随机导致单纯走法重放不稳定
工程可复现	Maven 多模块 + Fat JAR + <code>verify.ps1</code> 自检 + <code>demo.ps1</code> 演示 + Docker Compose 实验性部署 + Typst 文档体系，课程验收环境可快速构建、运行、测试和展示

2. 总体架构

2.1. 模块依赖

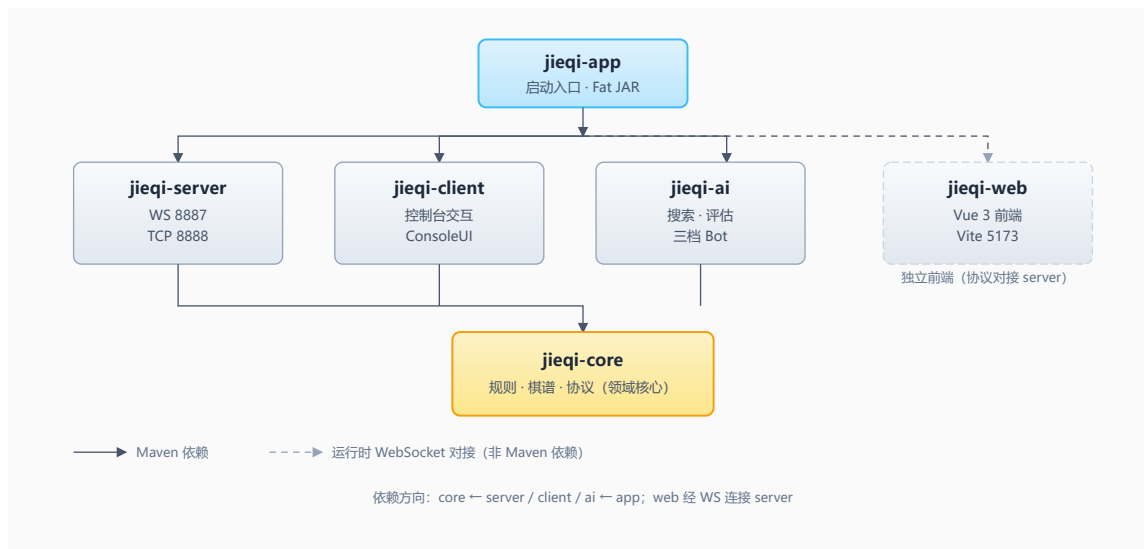


Figure 1: 模块依赖关系 (单向: core ← server/client/ai ← app; web 经 WS 对接 server)

2.2. 模块职责

模块	职责	不做什么
jieqi-core	棋盘、棋子、规则校验、终局判定、棋谱记法、协议模型	不含网络、AI、UI
jieqi-server	WS/TCP 服务、房间管理、匹配队列、用户注册、持久化、AI 调度	不含规则判断
jieqi-client	控制台交互、棋盘渲染 (10×9)、命令解析、复盘 n/p/g 命令	不含规则判断
jieqi-ai	AB 搜索、局面评估、Belief Sampling、三档 Bot、Agent 编排	不含领域逻辑
jieqi-app	统一启动菜单 (1-9 模式)、CLI 参数、Fat JAR 入口	无新业务逻辑
jieqi-web	Vue 3 对局界面、ChessBoard Canvas、聊天、终局查看、复盘控件	不含权威规则判定 (仅含前端提示用预校验)

架构层	职责与模式
通信层	- WsGameServer: WebSocket 连接管理、JSON 消息序列化/反序列化、心跳检测、断线重连检测 - GameServer: TCP 文本帧服务器 (\n 分隔)、粘包/半包帧解码 (FrameDecoder) - 模式: 外观 (WsGameServer 封装底层 WebSocket API)、单例 (全局服务实例)
消息处理层	- 消息路由: 按 messageType 分发至对应 Handler (handleLogin / handleMove / handleChat 等) - 校验链: 登录校验 → 房间校验 → 回合校验 → 规则校验 → 终局校验 - 模式: 策略 (消息类型到处理策略的映射)、命令 (chat / draw / resign / rematch 等指令对象)
业务逻辑层	- jieqi-core 领域模型: Board 聚合根 + ChessPiece 实体 + RuleValidator 领域服务 + EndgameJudge 领域服务 - jieqi-ai 博弈引擎: 三档 Bot (EasyRuleBot / AlphaBetaBot / BeliefAlphaBetaBot) - 模式: 工厂 (AiConfig.forLevel 创建 AI 实例)、模板方法 (AiBot 接口 → 三档实现)
数据持久层	- GameRecordStore: 棋谱文件 records/{gameId}.jieqi 落盘与读取 - ReplayRecordStore: ReplayFrame JSON 持久化 records/{gameId}.replay.json - UserRegistry: 内存用户注册表 (昵称、会话管理) - 模式: 仓库 (GameRecordStore 封装文件 IO)、序列化器 (JsonMapper、MoveNotation)

Table 1: 四层架构设计模式总览

2.3. 对局主流程

```
moveRequest 到达服务器
|
├─[1] 对局状态?          一否→ 拒绝
├─[2] 轮到你?           一否→ 拒绝
├─[3] 超时 65s?         一是→ TIMEOUT
├─[4] 原地翻子?         一是→ 拒绝
├─[5] 走法几何合法?     一否→ invalid
├─[6] 是否送将?         一是→ 拒绝
|
├─[7] Board.executeMove
├─[8] GameRecord.append
├─[9] ReplayTimeline.recordAfterMove
|
└─[10] EndgameJudge.checkAfterMove
    |
    └─ 终局 → gameOver + 落盘
    └─ 继续 → moveResult + 换方
```

Listing 2: 服务器走子校验与执行全链路（10 步）

设计模式	应用场景	具体实现
外观 (Facade)	WsGameServer 对外暴露单一入口	客户端只需连接 WebSocket URL，内部消息路由、校验、广播全部封装在 WsGameServer 内部。
单例 (Singleton)	全局唯一服务实例	GameServer / WsGameServer 各维护一个实例；UserRegistry 全局用户注册表。
工厂 (Factory)	AiConfig.forLevel() 创建 AI	根据 AiLevel (EASY/MEDIUM/HARD) 返回不同的 AiConfig 参数配置，创建对应的 Bot 实例。
策略 (Strategy)	消息类型 → 处理策略	messageType 字符串映射至对应 Handler 方法："chat" → handleChat(), "move" → handleMove(), "drawOffer" → handleDrawOffer()。
命令 (Command)	对局操作抽象	chat、drawOffer/resign/rematchRequest/addTime/pauseGame 等对局内操作以消息对象传递，服务端解析后执行对应命令逻辑。
模板方法 (Template)	AiBot 接口 → 三档 Bot	AiBot 接口定义 selectMove() 契约，EasyRuleBot / AlphaBetaBot / BeliefAlphaBetaBot 各自实现搜索策略。
观察者 (Observer)	broadcastRoom() 广播	走子后向房间内双方 + 观战者广播 moveResult/gameOver，客户端被动接收更新 UI。

Table 2: 通信层设计模式 — 外观 / 单例 / 工厂 / 策略 / 命令 / 模板方法 / 观察者

状态	触发条件	行为
Created	任一玩家发送 <code>createRoom</code> 或匹配队列新建房间	创建 Room 对象，分配 <code>roomId</code> ，返回 <code>joinCode</code> 供另一方加入。
Waiting	至少 1 名玩家已加入，等待对手	匹配队列轮询、邀请码加入、AI 对手加入；双方就绪后自动进入 Playing。
Playing	双方均已 Ready 且 <code>gameStart</code> 已广播	接受 <code>move/chat/drawOffer/resign/replayRequest/rematchRequest</code> 等消息；步时 65s 倒计时；终局条件触发后进入 GameOver。
GameOver	EndgameJudge 判定终局（吃将/将死/困毙/超时/认输/和棋/长将/长捉）	广播 <code>gameOver</code> 含终局原因与 <code>capturedReveal</code> ；落盘棋谱与 <code>replay.json</code> ；房间保留在内存供 Web 复盘； <code>rematchRequest</code> 可触发 <code>new gameStart</code> 回到 Playing。
Disconnected	玩家 WebSocket 断线超过容忍阈值	对方收到 <code>gameOver</code> (ABANDONED)，房间进入 GameOver 后清理。

Table 3: 房间状态机 — Created → Waiting → Playing → GameOver

2.4. 核心类图

Board	10×9 棋盘，棋子查询、make/undo 走子、AI 脱敏视角、局面哈希
ChessPiece	color、type（真实）、virtualType（原位角色）、revealed、row、col
Move	source、destination、isFlipOnly
Game	对局状态机：board、currentTurn、status、gameRecord、replayTimeline
RuleValidator	isValidMove / isMoveLegal / generateStrictLegalMoves
EndgameJudge	checkAfterMove：吃将→将死→困毙→40 步和→长将/长捉/超时/认输
GameRecord	文字棋谱 → *.jieqi；标准记法含回合号、红黑标识、翻子标记

ReplayTimeline	List<ReplayFrame> 快照时间线；每帧含 stepIndex/move/board/captured/status
AiBot	接口 $\leftarrow$ EasyRuleBot / AlphaBetaBot (JieqiAgent) / BeliefAlphaBetaBot
OptimizedAlphaBetaBot	搜索内核：AB 剪枝 + 迭代加深 + TT + 杀手/历史 + 静态搜索 + LMR

2.5. 坐标系统

棋盘为 10 行 $\times$ 9 列，坐标为“字母列 + 数字行”。

维度	范围	方向
行	0 - 9 (共 10 行)	从上到下：9 $\rightarrow$ 0
列	a - i (共 9 列)	从左到右：a $\rightarrow$ i

	a	b	c	d	e	f	g	h	i
9	車	馬	象	士	將	士	象	馬	車
8	•	•	•	•	•	•	•	•	•
7	•	炮	•	•	•	•	•	炮	•
6	卒	•	卒	•	卒	•	卒	•	卒
5	•	•	•	•	•	•	•	•	•
4	•	•	•	•	•	•	•	•	•
3	兵	•	兵	•	兵	•	兵	•	兵
2	•	炮	•	•	•	•	•	炮	•
1	•	•	•	•	•	•	•	•	•
0	車	馬	相	士	帥	士	相	馬	車

Table 4: 棋盘初始布局（行 0 = 红方底线，行 9 = 黑方底线）
 重要约定：先手（红方）始终在下方（行 0 - 4），后手（黑方）在上方（行 5 - 9）。
 坐标字符串中行号直接使用显示行号（如红帅 = "e0"，黑将 = "e9"）。内部数组转换：
`row = 9 - displayRow`, `col = coord.charAt(0) - 'a'`。

2.6. 虚拟类型机制

暗子未翻开时，其移动规则由所在位置的“虚拟类型”决定（该位置按中国象棋初始布局本应放置的棋子类型）。

	a	b	c	d	e	f	g	h	i
9	車	馬	象	士	將	士	象	馬	車
8	—	—	—	—	—	—	—	—	—
7	—	炮	—	—	—	—	—	炮	—
6	卒	—	卒	—	卒	—	卒	—	卒
5	—	—	—	—	—	—	—	—	—
4	—	—	—	—	—	—	—	—	—
3	兵	—	兵	—	兵	—	兵	—	兵
2	—	炮	—	—	—	—	—	炮	—
1	—	—	—	—	—	—	—	—	—
0	車	馬	相	士	帥	士	相	馬	車

Table 5: 棋盘各位置对应的虚拟类型（红底格为开局即明的将/帅）

2.7. 明子强化规则

暗子翻开为明子后，士和象获得强化：

棋子	暗子状态	明子状态（强化）
士 / 仕	斜走一格，限于九宫内	斜走一格，可离宫、可过河
象 / 相	田字走法，不可过河，塞象眼有效	田字走法，可过河，塞象眼不变

其他棋子走法规则与中国象棋完全一致。

2.8. 一次走子的完整调用链

以下跟踪一次走子请求从客户端到服务端再广播回所有客户端的完整路径。

Web 前端 / 控制台客户端

```

→ 发送 move JSON (fromX, fromY, toX, toY, isFlip)
→ WsGameServer.handleMove (WebSocket onMessage 分发)
→ JsonMessages.parseMove (JSON → Move 对象)
→ RandomRevealService.sanitizeClientMove (清除客户端伪造 type)
→ Game.processMove (对局状态机入口)
→ RuleValidator.isValidMove (几何走法合法?)
→ RuleValidator.isMoveLegal (不送将?)
→ Board.executeMove (走子 + 翻子 + 吃子 + 切换回合)
→ EndgameJudge.checkAfterMove (优先级判定：吃将→将死→困毙→无吃子和→重复→继续)
→ JsonMessages.moveResult / gameOver (构建响应)
→ WebSocket 广播 (moveResult / gameOver 推送给双方)

```

各步骤代码位置：WsGameServer.java L722 - 808 (handleMove 分发) • Game.java processMove (对局状态机) • RuleValidator.java (双层校验) • EndgameJudge.java (终局判定)。

3. WebSocket + JSON 通信协议规范

3.1. 基础约定

3.1.1. 术语定义

术语	含义
暗子	背面朝上、尚未翻开的棋子，按所在位置对应的中国象棋棋子规则移动
明子	正面朝上、已翻开的棋子，按实际类型规则移动
翻子	暗子完成合法移动或吃子后变为明子的揭示过程；标准规则不允许原地翻子
先手	红方，行棋优先权，棋盘下方
后手	黑方，棋盘上方
回合	一方完成一次合法走子
半步	一方的一次走子（40 回合 = 双方共 80 个半步）

3.1.2. 技术约定

项目	约定	备注
传输协议	WebSocket + JSON	课程公共接口；默认端口 8887
TCP 扩展	文本帧 msgType len payload\n	本组保留；默认端口 8888；见附录 A
字符编码	UTF-8	JSON 与 TCP payload 均为 UTF-8
WS 消息识别	messageType 字符串	每条 JSON 对象必含此字段
心跳	ping / pong，建议 10s	未实现心跳的客户端应被兼容
超时阈值	65 秒（60 秒思考 + 5 秒网络裕量）	服务器可配置

项目	约定	备注
时间戳单位	毫秒	<code>System.currentTimeMillis()</code> 风格
时间戳权威方	服务器	客户端时间戳仅供参考

3.2. 棋子类型编码

编码	类型	红方名	黑方名
0	KING	帅	将
1	ROOK	车	车
2	KNIGHT	马	马
3	CANNON	炮	炮
4	PAWN	兵	卒
5	ADVISOR	仕	士
6	BISHOP	相	象
-1	UNKNOWN	暗	暗

3.2.1. 老师 JSON piece 枚举映射

JSON 值	本组内部类型
king	KING（将/帅）
advisor / guard	ADVISOR（士/仕）
bishop	BISHOP（象/相）
rook	ROOK（车）
knight	KNIGHT（马）
cannon	CANNON（炮）
pawn	PAWN（兵/卒）
unknown	UNKNOWN（暗子未翻开）

3.2.2. 颜色编码

JSON 值	内部枚举	说明
red	RED	先手/红方

JSON 值	内部枚举	说明
black	BLACK	后手/黑方

3.3. Move 对象规范

3.3.1. 字段规则

字段	规则
fromX, fromY	源坐标; fromX=a-i, fromY=0-9
toX, toY	目标坐标; 范围同上
isFlip	布尔值; true 表示本步会翻开棋子 (暗子首动); source == destination 为原地翻子 (服务器拒绝)

3.3.2. 翻子随机性机制

- 暗子真实 piece 类型在 Board.initBoard() 阶段已通过 Collections.shuffle 随机分配并写入棋盘; RandomRevealService 仅负责清除客户端伪造 type, 走子后写回服务端真实类型, 不在此步重新随机
- 客户端不可预测或控制翻子结果, move 请求中不指定预期类型
- moveResult.flipResult 包含 { x, y, piece }, 客户端据此更新本地棋盘

3.3.3. JSON move 对象格式

move (C→S, 含翻子)

```
{
  "fromX": "b",
  "fromY": 7,
  "toX": "b",
  "toY": 4,
  "isFlip": true
}
```

moveResult (S→C, valid=true; captured 按红/黑视角差异化)

```
{
  "messageType": "moveResult",
  "roomId": "abc123",
  "valid": true,
  "move": { "fromX": "b", "fromY": 7, "toX": "b", "toY": 4, "isFlip": true },
  "flipResult": { "x": "b", "y": 4, "piece": "cannon" },
  "captured": { "color": "red", "piece": "pawn", "wasDark": false },
  "currentTurn": "black",
  "moveTime": 3421
}
```

3.4. 完整消息类型规范

3.4.1. 客户端 → 服务器消息

messageType	说明	本组实现
Login	登录认证, 含 userId/password/nickname/avatar	已实现
register	注册新用户	已实现
startMatch	发起随机匹配	已实现
cancelMatch	取消匹配	已实现
startAiGame	发起人机对弈, 含 aiLevel (easy/medium/hard)	已实现
startAiBattle	AI 自动对弈观战, 含 aiLevel1/aiLevel2	已实现
createRoom	创建房间	已实现
joinRoom	加入房间, 含 roomId	已实现
requestFirstHand	争先手 (true = 要先手 / false = 要后手)	已实现
Ready	准备就绪	已实现
move	走子请求, 含 fromX/fromY/toX/toY/isFlip	已实现
chat	聊天 (≤ 120 字符)	已实现
Resign	认输	已实现
drawOffer	提和	已实现
drawAccept	同意提和	已实现
drawDecline	拒绝提和	已实现
rematchRequest	请求重赛	已实现
rematchDecline	拒绝重赛	已实现
replayRequest	请求复盘帧, 可选 stepIndex (默认最后一帧)	已实现
addTime	手动加时 (每步最多 2 次, 每次 +60s)	已实现
pauseGame	暂停对局 (仅 AI 模式)	已实现
resumeGame	恢复对局	已实现
watch	观战请求, 含 roomId	已实现
ping	心跳保活, 含 timestamp	已实现

Login

```
{
  "messageType": "Login",
  "userId": "player1",
  "password": "123456",
  "nickname": "隐形小炮兵",
  "avatar": "avatar_01"
}
```

startAiGame

```
{
  "messageType": "startAiGame",
  "aiLevel": "medium"
}
```

move (含翻子)

```
{
  "messageType": "move",
  "fromX": "b",
  "fromY": 7,
  "toX": "b",
  "toY": 4,
  "isFlip": true
}
```

3.4.2. 服务器 → 客户端消息

messageType	说明	本组实现
loginResult	登录结果, 含 success/userId/nickname/avatar	已实现
matchSuccess	匹配成功, 含 roomId/opponentId/opponentNickname/opponentAvatar	已实现
roomInfo	房间状态广播, 含 opponentReady	已实现
gameStart	对局开始, 含 redPlayerId/blackPlayerId/yourColor/firstHand/initialBoard	已实现
moveResult	走子结果, 含 valid/move/flipResult/captured/currentTurn	已实现
flipResult	翻子结果 (单独消息兼容)	已实现
gameOver	终局, 含 winner/reason/winnerId/capturedReveal	已实现
timeout	超时通知, 含 loserId	已实现
chatMessage	聊天广播, 含 fromUserId/content/timestamp	已实现

messageType	说明	本组实现
drawOffered	提和通知	已实现
drawDeclined	拒和通知	已实现
rematchOffer	重赛邀请	已实现
rematchDeclined	重赛拒绝	已实现
replayFrame	复盘帧, 含 stepIndex/totalSteps/board/ move/captured/currentTurn/status	已实现
timeBonus	加时结果, 含 turnStartTime/forColor	已实现
gamePaused	暂停通知	已实现
gameResumed	恢复通知	已实现
pong	心跳响应	已实现
error	错误通知, 含 errorCode/message	已实现

loginResult

```
{
  "messageType": "loginResult",
  "success": true,
  "userId": "player1",
  "nickname": "隐形小炮兵",
  "avatar": "avatar_01"
}
```

matchSuccess

```
{
  "messageType": "matchSuccess",
  "roomId": "1ca389",
  "opponentId": "ai-bot-medium",
  "opponentNickname": "标准 AI",
  "opponentAvatar": "bot"
}
```

gameStart

```
{
  "messageType": "gameStart",
  "roomId": "1ca389",
  "redPlayerId": "player1",
  "blackPlayerId": "ai-bot-medium",
  "yourColor": "red",
  "firstHand": true,
  "initialBoard": [
    { "x": "a", "y": 0, "color": "red", "piece": "rook", "visible": false },
    { "x": "e", "y": 0, "color": "red", "piece": "king", "visible": true },
    ...    // 共 32 个棋子
  ]
}
```

```
]
}
```

moveResult (valid=true, 含 captured 信息差)

```
{
  "messageType": "moveResult",
  "roomId": "1ca389",
  "valid": true,
  "move": { "fromX": "b", "fromY": 7, "toX": "b", "toY": 4, "isFlip": true },
  "flipResult": { "x": "b", "y": 4, "piece": "cannon" },
  "captured": { "color": "red", "piece": "pawn", "wasDark": false },
  "currentTurn": "black"
}
```

moveResult (valid=false, 非法走法)

```
{
  "messageType": "moveResult",
  "roomId": "1ca389",
  "valid": false,
  "message": "非法走法"
}
```

gameOver (capturedReveal 揭晓全部暗子身份)

```
{
  "messageType": "gameOver",
  "roomId": "1ca389",
  "winner": "black",
  "reason": "checkmate",
  "winnerId": "ai-bot-medium",
  "capturedReveal": [
    { "color": "red", "piece": "rook", "wasDark": true },
    { "color": "black", "piece": "cannon", "wasDark": true },
    ...
  ]
}
```

error

```
{
  "messageType": "error",
  "errorCode": 2001,
  "message": "非法走法"
}
```

3.4.3. 公共数据结构

initialBoard 数组: 每元素 { x: "a"-"i", y: 0-9, color: "red"|"black", piece: "rook"|..., visible: true|false }。visible = 是否明子。

3.4.4. 游戏结果原因 (gameOver.reason)

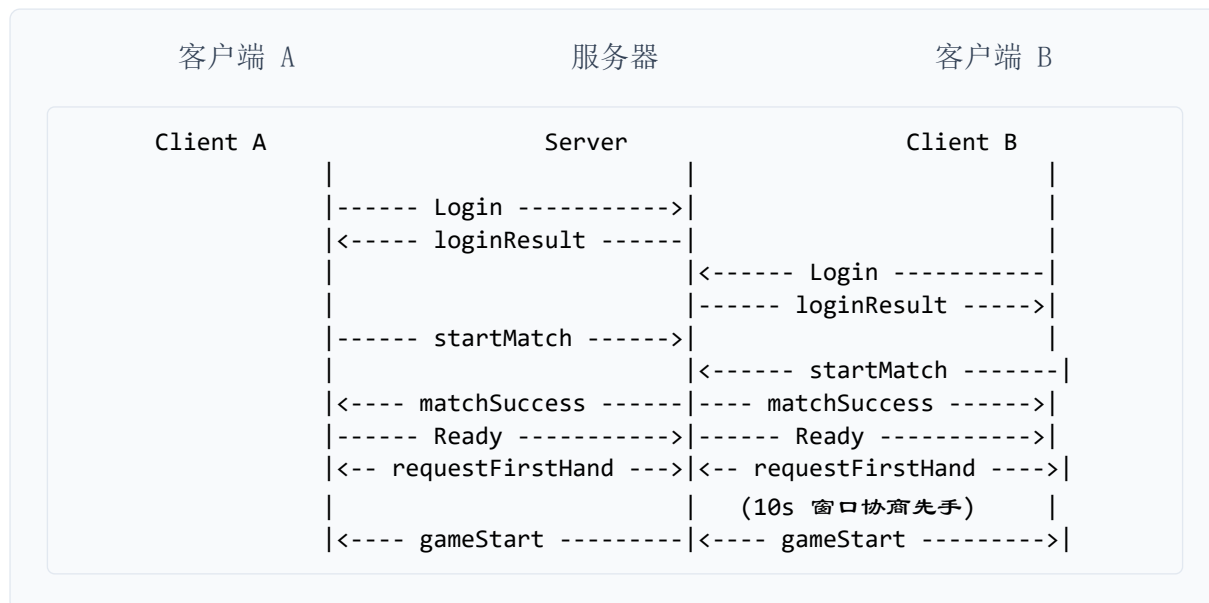
reason	说明
checkmate	将死 — 被将军且无合法走法
stalemate	困毙 — 无合法走法且未被将军
timeout	超时 — 单步超过 65 秒
resign	认输 — 玩家主动认输
disconnect	断线 — WebSocket 意外关闭
ruleViolation	规则违例 — 长将判负等
abandoned	放弃 — 离开房间
draw	和棋 — 40 步无吃子 / 双方同意

3.4.5. 错误码

错误码	含义
1001	登录失败 — 用户不存在或密码错误
2001	非法走法 — 棋子规则不允许
2002	送将 — 走子后己方被将军
3001	匹配失败 — 已在匹配/房间中
3002	房间不存在或已满
4001	超时局终
5001	消息格式解析失败 / 未知消息类型

3.5. 典型通信时序

3.5.1. 匹配与先手协商



Listing 3: 登录 → 匹配 → Ready → 先手协商 → 开局


```
sequenceDiagram
    participant A as Client A (Red)
    participant S as Server
    participant B as Client B (Black)
    A->>S: move b7→b4 (isFlip:true)
    S-->>A: moveResult flipResult: cannon
    S->>B: move h6→h4 (isFlip:true)
    B-->>S: moveResult flipResult: knight
    S-->>A: moveResult ... loop until gameOver ...
```

The diagram illustrates the interaction between three entities: Client A (Red), Server, and Client B (Black). The sequence of messages is as follows:

- Client A (Red) sends a message to the Server: `move b7→b4 (isFlip:true)`.
- The Server returns a message to Client A (Red): `moveResult flipResult: cannon`.
- The Server sends a message to Client B (Black): `move h6→h4 (isFlip:true)`.
- Client B (Black) returns a message to the Server: `moveResult flipResult: knight`.
- The Server returns a message to Client A (Red): `moveResult ... loop until gameOver ...`.

3.5.3. 非法着法



3.5.4. 超时判负



21

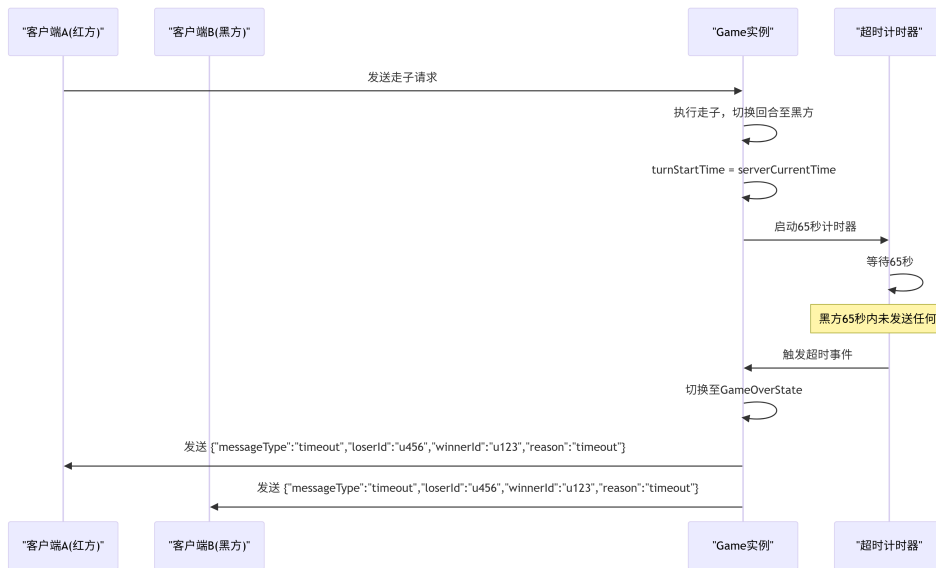
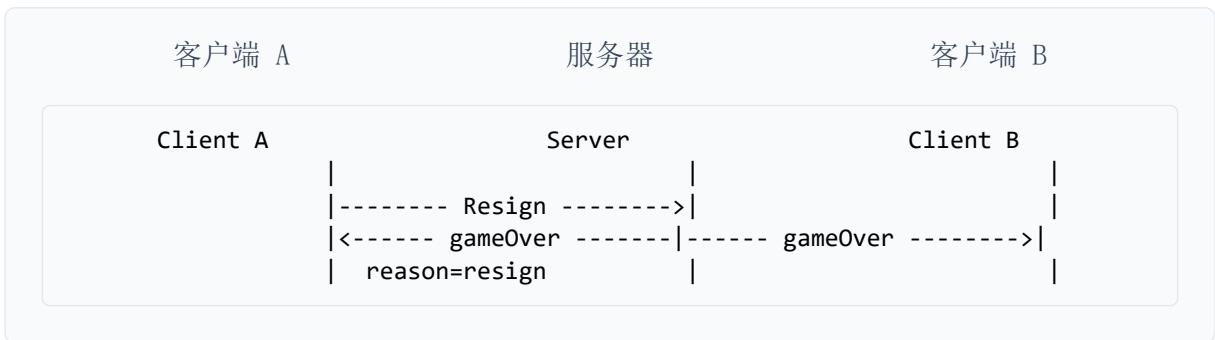


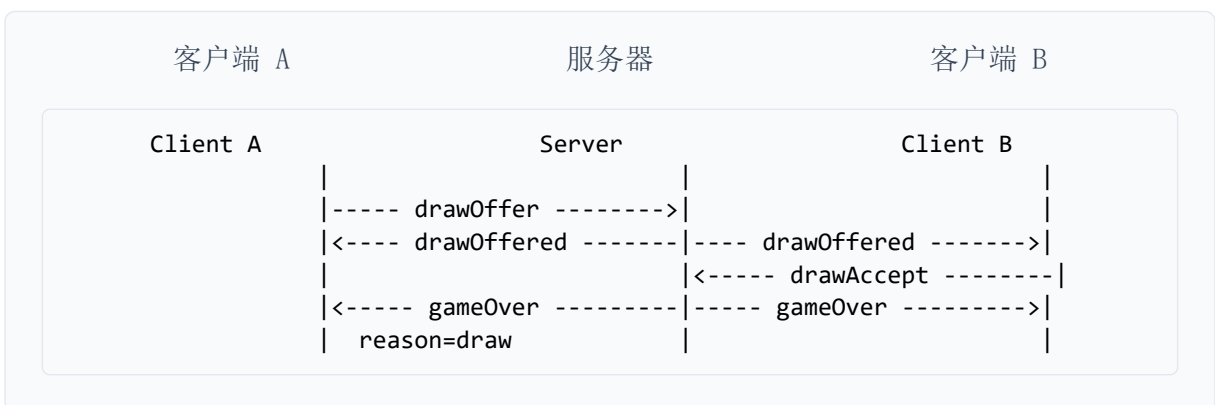
Figure 3: 65 秒步时超时机制 — 定时器、超时检测、gameOver 广播全链路

3.5.5. 认输



Listing 7: 认输: 广播 gameOver (reason=resign)

3.5.6. 提和流程



Listing 8: 提和: A 提和 → B 同意 → gameOver (reason=draw)

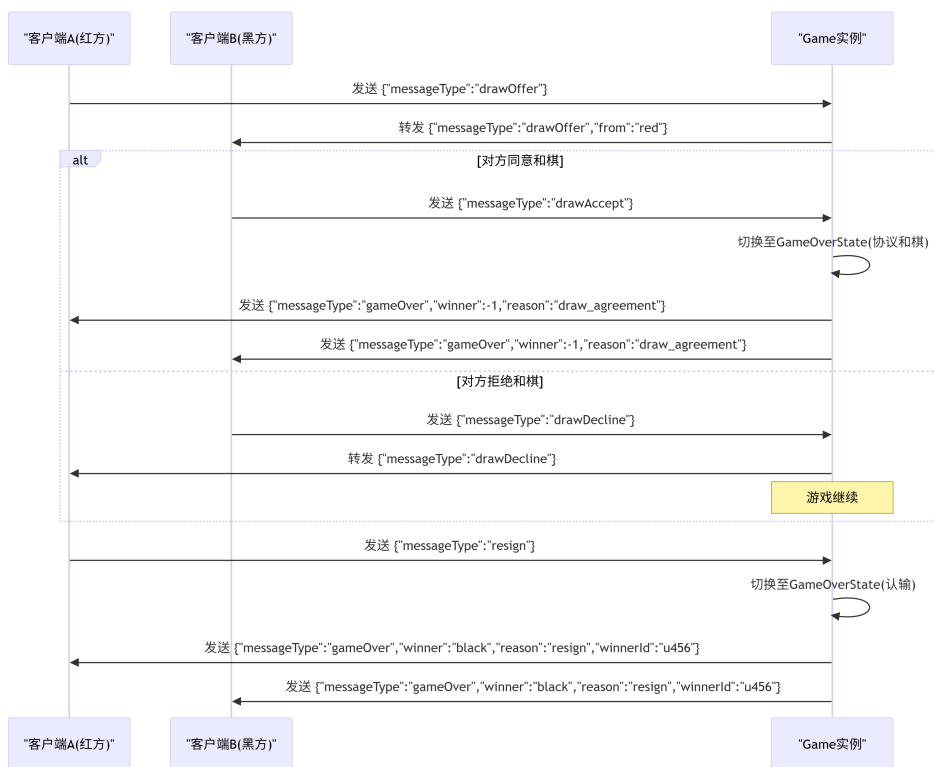
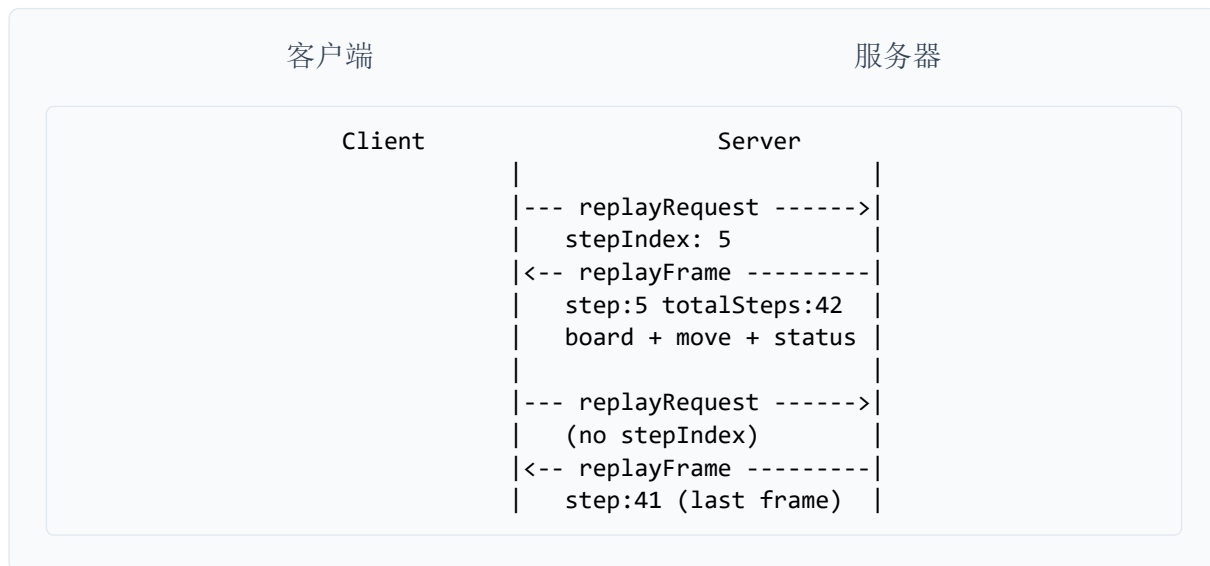


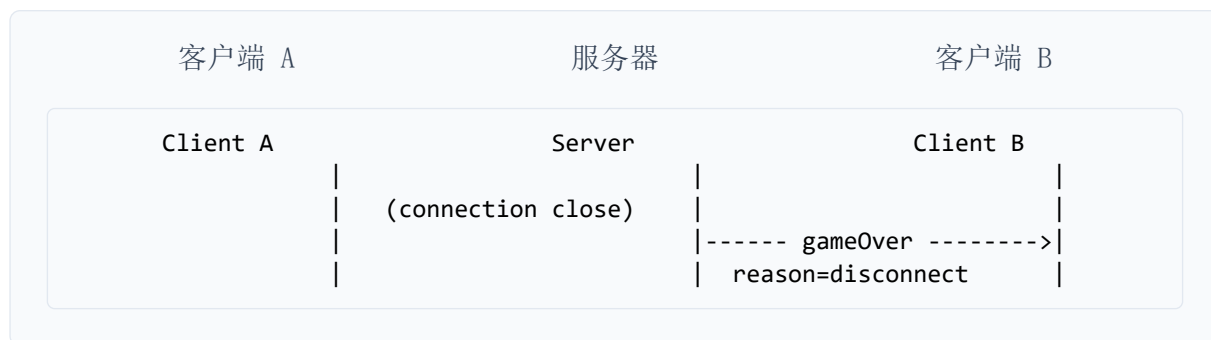
Figure 4: 提和与认输流程 — drawOffer / drawAccept / drawDecline / resign 时序

3.5.7. 复盘请求



Listing 9: 复盘: replayRequest → server 查 timeline → replayFrame (每帧带完整 board 快照)

3.5.8. 断线判负



Listing 10: 一方 WebSocket 关闭：对方收到 gameOver (reason=disconnect)

3.6. 本组扩展消息总览

除课程公共消息外，本组额外扩展以下消息类型：

messageType	方向	说明
startAiGame	C→S	人机对弈，含 aiLevel (easy/medium/hard)
startAiBattle	C→S	AI 自动对弈观战
replayRequest	C→S	请求复盘帧，可选 stepIndex
replayFrame	S→C	复盘帧：stepIndex/totalSteps/board/move/captured
rematchRequest	C→S	请求重赛
rematchOffer / rematchDeclined	S→C	重赛邀请/拒绝
addTime	C→S	手动加时 +60s（每步最多 2 次）
timeBonus	S→C	加时结果通知
watch	C→S	观战请求
pauseGame / resumeGame	C→S	暂停/恢复（AI 模式）
gamePaused / gameResumed	S→C	暂停/恢复通知

3.7. 协议实现状态

以下对照课程公共协议 v3.1 列出每条消息的代码实现状态。**公共协议** 表示 INTERFACE.typ 规定的标准消息；**本组扩展** 表示为支持复盘、重赛等功能新增的消息类型。

消息类型	协议来源	服务端	控制台	Web	说明
login	公共协议	已实现	已实现	已实现	Web 额外支持昵称与预设头像
startMatch	公共协议	已实现	已实现	已实现	真人对战匹配
startAiGame	公共协议	已实现	已实现	已实现	人机三档难度选择
move	公共协议	已实现	已实现	已实现	服务端权威校验，客户端仅交互预检
moveResult	公共协议	已实现	已实现	已实现	含 flipResult + captured
gameOver	公共协议	已实现	已实现	已实现	含 capturedReveal 终局揭晓
chat / chatMessage	公共协议	已实现	已实现	已实现	≤ 120 字符校验，仅真人可用
drawOffer / drawAccept	公共协议	已实现	已实现	已实现	双方确认和棋
resign	公共协议	已实现	已实现	已实现	主动认输
timeout	公共协议	已实现	已实现	已实现	服务端 65s 步时计时器
replayRequest / replayFrame	本组扩展	已实现	已实现	已实现	仅终局后开放；Web 复盘依赖房间在内存中
rematchRequest / rematchOffer	本组扩展	已实现	已实现	已实现	双方同意后新 gameStart
addTime	本组扩展	已实现	已实现	已实现	手动加时 +30s，每局限 2 次
pauseGame / resumeGame	本组扩展	已实现	已实现	已实现	AI 模式专用
watch	本组扩展	未实现	已实现	部分实现	控制台观战已实现；Web 待实现

4. 规则引擎设计

规则引擎位于 `jieqi-core` 模块，核心类包括 `Board`、`ChessPiece`、`Move`、`RuleValidator` 和 `Game`。`Board` 负责维护棋盘、棋子列表、走法执行、吃子和回滚；`ChessPiece` 负责保存棋子的真实类型、虚拟类型、颜色和翻开状态；`RuleValidator` 负责判断单步走法是否合法；`Game` 负责对局状态、回合切换、超时、棋谱、复盘和终局判定。

揭棋与标准象棋最大的区别在于：除将帅外，大部分棋子开局为暗子。暗子具有两个类型字段：`type` 表示真实类型，`virtualType` 表示当前位置对应的虚拟走子类型。未翻开的暗子按照 `virtualType` 生成走法，移动后翻开并显示真实 `type`。因此，揭棋同时具有“标准象棋走法约束”和“暗子信息不完全”的特点。

4.1. 七种棋子走法速查

`RuleValidator` 的走法判断流程为：先根据坐标找到源棋子和目标格，确认源棋子存在、颜色属于当前行棋方、目标格不是己方棋子；然后拒绝原地翻子；最后通过 `piece.getMoveType()` 取得当前应使用的走法类型——对于暗子返回 `virtualType`，对于明子返回真实 `type`。因此，暗子在未翻开前不会暴露真实身份。

棋子	走法规则	暗/明差异
车 / 俥	同行或同列直线移动，路径中间不能有棋子；吃子时目标格必须为对方棋子	无差异
马 / 傌	走日字 ($\pm 2, \pm 1$) 或 ($\pm 1, \pm 2$)；前进方向相邻格有子则蹩马腿，禁止跳跃	无差异
炮 / 砲	平移时路径中不能有棋子；吃子时路径中必须恰好有一个炮架；无炮架不能吃子	无差异
兵 / 卒	未过河只能向前走 1 格；过河后可向前或左右平移 1 格；不能后退	无差异

棋子	走法规则	暗/明差异
将 / 帅	在九宫内上下左右移动 1 格；不能造成将帅照面	开局即为明子
士 / 仕	斜走 1 格	暗士限九宫；明士可全场斜走，不再受九宫限制
象 / 相	走田字，即斜向移动 2 格；中点有子则塞象眼，禁止移动	暗象不过河；明象可过河，但仍受塞象眼限制

4.2. 暗子特殊规则

规则	说明
走法按 <code>virtualType</code>	暗子不暴露真实身份，走法按照当前位置对应的虚拟类型生成。例如底线车位暗子即使真实身份不是车，未翻开前仍按车的走法移动
真实类型开局已随机分配	棋盘初始化时， <code>Board.initBoard()</code> 对双方 15 个暗子池做 <code>Collections.shuffle</code> 洗牌，真实 <code>type</code> 已写入棋盘的 <code>ChessPiece.type</code> 字段
客户端不能指定翻子结果	客户端上传的 <code>type</code> 会被 <code>RandomRevealService</code> 清除；最终翻开什么由服务端棋盘中的真实 <code>type</code> 决定； <code>RandomRevealService</code> 不在此步重新随机分配
首次移动后翻开	暗子一旦完成合法移动， <code>revealed = true</code> ，显示其真实 <code>type</code>
暗士限九宫	未翻开的士按照标准象棋士的规则移动，只能在本方九宫内斜走 1 格
明士可出九宫	士翻开后获得强化效果，可在全棋盘范围内斜走 1 格
暗象不过河	未翻开的象按照标准象棋象的规则移动，走田字、受塞象眼限制，并且不能过河
明象可过河	象翻开后获得强化效果，可以过河，但仍然必须走田字，且仍然受塞象眼限制
禁止原地翻子	<code>source == destination</code> 或 <code>isFlipOnly</code> 时服务器拒绝；正式对局入口 <code>Game.processMove</code> 阻止原地翻子

规则	说明
	(<code>Board.executeMove</code> 中仅保留兼容早期测试的 <code>flipOnly</code> 分支)
无吃子移动	普通移动若未吃子则递增 <code>noCaptureCount</code> (每次移动 +1); 发生吃子则 <code>noCaptureCount = 0</code> (<code>noCaptureCount ≥ 80</code> 时判和, 即 80 个半步无吃子、双方合计 40 回合)

4.3. 暗子真实类型与虚拟类型

每个暗子同时包含两个类型, 外加翻开状态:

字段	含义	作用
<code>type</code>	真实类型	暗子翻开后的实际棋子身份; 开局已随机分配
<code>virtualType</code>	虚拟类型	暗子未翻开时用于生成走法; 由开局位置决定
<code>revealed</code>	是否翻开	决定当前走法使用 <code>type</code> 还是 <code>virtualType</code>

未翻开时 `moveType = virtualType`, 翻开后 `moveType = type`。

例如, 一个位于车位的暗子, `virtualType` 是 `ROOK`, 因此未翻开前按车走; 但它的真实 `type` 可能是 `CANNON`, 移动后翻开就会变成炮。这个机制是揭棋的核心不确定性来源, 也是 AI 不能直接透视对手暗子的原因, 以及复盘必须保存棋盘快照而非仅走法文本的根本原因。

4.4. 服务端权威校验

对局中, 客户端只负责发送走法, 最终是否合法由服务器判断。服务端收到 `move` 消息后, 会先解析源坐标和目标坐标, 然后清除客户端可能伪造的棋子类型, 再交给 `Game.processMove` 处理:

```

Game.processMove
├─[1] 对局状态 == PLAYING?    一否→ 拒绝
├─[2] 轮到当前玩家?          一否→ 拒绝
├─[3] 已超时? (65s)          一是→ 终局: TIMEOUT
├─[4] 原地翻子?              一是→ 拒绝
├─[5] RuleValidator.isValidMove 一否→ 拒绝: 非法走法 (2001)
├─[6] RuleValidator.isMoveLegal 一否→ 拒绝: 不能送将 (2002)
├─[7] Board.executeMove      ← 执行走子 / 翻子 / 吃子
├─[8] GameRecord.append      ← 文字棋谱追加
├─[9] ReplayTimeline.record ← 复盘快照
├─[10] EndgameJudge.checkAfterMove
    ├ 有终局 → gameOver 广播 + 棋谱/replay 落盘
    └ 无终局 → moveResult 广播 + 轮次切换

```

Listing 11: 服务端走子校验全链路 — 客户端不能通过伪造 type、伪造翻子结果或发送非法坐标改变服务器权威棋局

这种设计保证了客户端无法通过伪造 type、伪造翻子结果或发送非法坐标来改变服务器权威棋局。

4.5. 将军、送将与将帅照面

普通几何走法合法并不代表最终合法。系统还会通过试走方式判断走完后己方是否被将军。如果某一步虽然满足棋子走法，但会导致己方将帅被攻击，系统会拒绝该走法并返回“不能送将”。

将帅照面也在规则引擎中处理：如果双方将帅位于同一列，且中间没有任何棋子阻挡，则认为当前局面存在将帅照面风险。相关判断被纳入 `isInCheck`，从而影响合法走法生成、将军判断和终局判定。

4.6. 校验流程

两层校验职责分离——这是标准象棋引擎的 `generate-and-test` 范式，搜索树与服务器校验共用同一套逻辑：

方法	检查内容	不检查
<code>isValidMove</code>	几何走法合法性、阵营归属、棋子类型规则	是否送将
<code>isMoveLegal</code>	试走一子后己方是否被将军（make/unmake 模拟）	—

方法	检查内容	不检查
<code>generateStrictLegalMoves</code>	枚举全部几何走法 → <code>makeMove</code> → <code>isInCheck</code> → <code>undoMove</code> , 过滤送将步	—

局面哈希: `Board.positionKey(board, sideToMove)` 采用字符串键 (10×9 格子 + | + `sideToMove`, 暗子用 ? 占位), 用于长将/长捉重复局面计数。发生吃子时清空所有 `repetitionCount`。AI 置换表 (TT) 则使用独立的 64-bit Zobrist XOR 哈希 (`ZobristHash.computeHash`, 仅 `jieqi-ai`), 两者并行存在、尚未统一。

4.7. 规则引擎设计特点

1. 走法与对局状态分离: `RuleValidator` 只负责判断单步规则, `Game` 负责回合、超时、棋谱、终局和复盘
2. 暗子走法统一抽象: 外部规则判断不需要关心棋子是否翻开, 只需调用 `getMoveType()`, 由 `ChessPiece` 内部决定返回 `virtualType` 还是 `type`
3. 服务端权威翻子: 客户端不能决定暗子翻开后的真实身份, 服务器清除客户端上传的类型并以棋盘真实状态为准
4. 强化规则局部化: 士和象的强化逻辑集中在 `RuleValidator` 的对应方法中, 暗子走标准规则、明子走强化规则, 便于测试和维护
5. 支持 AI 搜索与回滚: `Board` 提供 `makeMove` / `unmakeMove` 快照机制, 供合法走法生成和 AI 搜索使用, 避免搜索过程污染真实棋局

4.8. 终局判定

入口: `EndgameJudge.checkAfterMove`, 在 `Board.executeMove` 与棋谱记录之后调用。按固定优先级依次判定, 返回 `null` 表示对局继续。

```

吃子发生 → 清空 repetitionCount
├─ 吃掉明将？          → 走子方胜 (KING_CAPTURED)
├─ 对方被将死？        → 走子方胜 (CHECKMATE)
├─ 对方困毙？          → 走子方胜 (STALEMATE)
├─ noCaptureCount >= 80？ → 和棋 (NO_CAPTURE_DRAW)
├─ 更新 repetitionCount[boardHash]
├─ 重复 >= 6 次？
│   ├─ 将军中 → 长将判负
│   ├─ 兵卒长捉 → 和棋
│   └─ 长捉 → 判负
└─ 继续对弈

```

Listing 12: 终局判定优先级流程

重复计数维护：每步 $\text{count} = \text{repetitionCount}[\text{key}] + 1$ ；发生吃子 → 清空全部重复计数（局面本质变化）。

长捉检测（**findChaseTarget**）：走子后，若 destination 上的棋子可对某一方子生成合法吃子走法 → 视为“捉”。当前实现用 isValidMove 近似“捉”，尚未区分将/杀/捉、隔子捉/连环捉等裁判细节。

将军检测（**isInCheck**）：枚举对方所有子的几何攻击 + 将帅照面（同列无遮挡）。

4.9. 测试覆盖

领域	测试类	状态	说明
七种走法	BoardMakeMoveTest、 DarkPieceRuleTest	已实现	含暗士暗象 + 明士明象强化
送将 / 照面 / 解将	RuleEdgeCaseTest (11 项矩阵)	已实现	—
将死 / 困毙 / 吃将	EndgameJudgeTest、 RuleEdgeCaseTest	已实现	—
40 步无吃子	RuleEdgeCaseTest	已实现	—
undo / 搜索一致性	BoardUndoTest、 aiSearchMakeUnmakePreservesBoardHash	已实现	make/unmake 后 hash 不变
纯重复无将军无捉	EndgameJudgeTest.aiSearchMakeUnmakePreservesBoardHash	已实现	6 次重复不终局

领域	测试类	状态	说明
长将	RuleEdgeCaseTest	部分实现	主路径已实现；缺真实对局集成测
长捉	RuleEdgeCaseTest	部分实现	3 个局面通过；将/杀/捉未细分
错误原因码	—	未实现	RuleValidator 返回 boolean，未细化

P0 规则主路径（走子、送将、照面、将死/困毙、40 步和）可标 **已实现** 已实现。长将/长捉标 **部分实现** 是准确的——不是没做，而是裁判粒度不够细（见 § 13 已知限制与路线图）。

5. AI 算法设计

5.1. 问题形式化

揭棋在算法上是一个有限步、双人对弈、不完全信息的零和博弈：

信息类型	内容
完全信息	明子真实身份、将帅位置、已翻开历史
不完全信息	对手 15 枚暗子的真实 type（仅知 virtualType，即开局原位角色）

状态空间定义：

状态	10×9 棋盘 + (color, type, virtualType, revealed) 每子 + 轮次方 + noCaptureCount + repetitionCount
目标	在步时约束（约 60s + 5s 裕量）内输出严格合法走法；Hard 档还需在不透视对手暗子前提下近似最优

5.2. 规则引擎算法（RuleValidator + Board）

5.2.1. 两层合法性校验

采用几何合法与战术合法分离的标准象棋引擎 **generate-and-test** 范式，搜索树与服务器校验共用同一套逻辑：

层次	方法	检查内容
L1	isValidMove	阵营归属、目标格可达性、七种棋子几何走法

层次	方法	检查内容
L2	<code>isMoveLegal</code>	试走一子后己方是否被将军（禁送将）—— <code>make/unmake</code> 模拟
—	<code>generateStrictLegalMoves</code>	对所有几何走法逐一 <code>makeMove</code> → <code>isInCheck</code> → <code>undoMove</code> ，过滤送将步

5.2.2. 暗子走法依据：`getMoveType()`

<code>revealed == false</code>	按 <code>virtualType</code> （原位角色）生成走法
<code>revealed == true</code>	按真实 <code>type</code> 生成走法

5.2.3. 揭棋特有规则（算法分支）

暗士 / 暗象（标准象棋约束）：

- 暗士：斜走 1 格，限九宫内
- 暗象：田字走，不可过河，塞象眼有效

明士 / 明象（强化规则）：

- 明士：斜走 1 格，可出九宫（全场移动）
- 明象：田字走，可过河，塞象眼不变

翻子机制（`Board.executeMove`）：

- 暗子首次移动或吃子 → `revealed = true`
- 真实 `type` 在开局初始化时已随机分配：`Collections.shuffle` 对 15 子池洗牌后按格填入，走子时不重新随机
- `RandomRevealService` 仅做服务器权威校验：清除客户端伪造 `type`，走子后写回真实 `type`

局面哈希（`Board.positionKey`）：

- 编码：10×9 格子 + 待走方
- 暗子以 ? 占位，不泄露真实身份
- 用于长将/长捉重复局面判定与 AI 长将规避

5.3. 终局判定算法（`EndgameJudge`）

走子后按固定优先级依次判定，返回 `null` 表示对局继续：

优先级	结果	判定条件
1. 吃将	<code>KING_CAPTURED</code> → 走子方胜	对方将/帅被吃
2. 将死	<code>CHECKMATE</code> → 走子方胜	<code>isInCheck</code> ∧ 无合法解将步
3. 困毙	<code>STALEMATE</code> → 走子方胜	<code>!isInCheck</code> ∧ 无任何合法步

优先级	结果	判定条件
4. 无吃子和	和棋	<code>noCaptureCount</code> ≥ 80 (80 个半步无吃子, 即双方合计 40 回合无吃子判和)
5. 重复局面	见下文	同一 <code>positionKey</code> 累计次数 ≥ 6

重复局面判定 (`positionKey` 计数 ≥ 6 时):

- 仍在将军 $\rightarrow$ 将军方判负 (长将)
- 未将军但合法捉子 $\rightarrow$ 走子方判负 (长捉); 兵卒长捉 $\rightarrow$ 和棋

重复计数维护:

- 每步 `count = repetitionCount[key] + 1`
- 发生吃子 $\rightarrow$ 清空全部重复计数 (局面本质变化)

长捉检测 (`findChaseTarget`): 走子后, 若 `destination` 上的棋子可对某一方子生成合法吃子走法 $\rightarrow$ 视为「捉」。

将军检测 (`isInCheck`): 枚举对方所有子的几何攻击 + 将帅照面 (同列无遮挡)。

5.4. AI 三档架构

档位	实现类	核心算法	时间预算
Easy	<code>EasyRuleBot</code>	启发式排序 + Top-K 随机	$\leq 500\text{ms}$
Medium	<code>AlphaBetaBot</code> $\rightarrow$ <code>JieqiAgent</code>	公开视角 Alpha-Beta + Agent 编排	可配置, 默认 5s
Hard	<code>BeliefAlphaBetaBot</code>	Belief Sampling + 多次 Alpha-Beta 期望	5s (默认 4 采样 $\times$ 6 候选)

5.4.1. Easy: 启发式 + 随机

1. `createAiPublicView(color)` 脱敏
 2. `generateLegalMoves` $\rightarrow$ `MoveOrderer.sortByHeuristic` (MVV-LVA 排序)
 3. 30% 全随机; 70% 从 Top-8 中随机
- 无搜索树, 保证毫秒级响应与合法输出。

5.4.2. Medium: Agent 编排 + Alpha-Beta

`AgentOrchestrator` 按优先级串行调度三个子 Agent:

优先级	Agent	职责
10	<code>ProbabilityAgent</code>	设置 <code>evalBias</code> 注入评估偏置, 不直接选着
20	<code>EndgameAgent</code>	子力 ≤ 12 时接管搜索, 加深时间上限至 30s

优先级	Agent	职责
100	SearchAgent	默认主搜索（Alpha-Beta + 迭代加深）

信息约束： 所有搜索在 `createAiPublicView` 脱敏棋盘上进行——己方暗子保留真实 type，对手暗子 type = UNKNOWN。

5.5. Alpha-Beta 搜索引擎（OptimizedAlphaBeta）

5.5.1. 框架：Negamax + 迭代加深（ID）

迭代加深主循环

```
for depth = 1 .. MAX_DEPTH(20):
  aspiration search(depth, window = bestScore ± 80)
  if fail-low / fail-high → 全窗口重搜
  if 剩余时间 < 2 × 上一层耗时 → 停止加深
return 已完成层的最优走法
```

Anytime 特性： 任一层超时即返回上一层结果，避免空步响应。

5.5.2. 剪枝与加速技术

技术	实现要点
Alpha-Beta 剪枝	标准 negamax, α / β 窗口传递
Aspiration Window	± 80 缩窗；失败时扩至 $[-\text{INF}, +\text{INF}]$
PVS	首变全窗口；后续 null-window 试探 + 推高后重搜
LMR 晚着减少	quiet move + searched $\geq 4 + \text{depth} \geq 3 + \text{非 PV}$ → depth - 1
置换表 TT	Zobrist 64-bit 哈希, 2^{20} 槽位替换；存 (depth, score, flag, bestMove)；flag: EXACT / ALPHA / BETA
Killer Heuristic	每层记录 2 个 β -cutoff 走法，兄弟节点优先尝试
History Heuristic	推高 α 的走法累积历史分值；每 4 层 aging 衰减
Move Ordering	TT best → MVV-LVA → 翻子/暗子 → killer → history → 中心偏好
静态搜索 Quiescence	叶子 depth = 0 后仅扩展吃子，最多 3 层
SEE 过滤	静态搜索中 SEE < 0 的亏交换直接跳过
长将规避	repetition[key] + 1 ≥ 5 且仍在将军 → 该步 score - 100000

5.5.3. 静态交换评估 SEE (StaticExchangeEvaluator)

对给定吃子着法，在目标格模拟双方依次用最便宜攻击子互吃的序列，反向 minimax 求当前方净收益：

SEE > 0	赚的交换，优先搜索
SEE = 0	等值交换
SEE < 0	亏的交换（如车换卒），quiescence 中跳过

不精确处理炮的 X-ray 遮挡，但对明显亏交换足够有效。

5.5.4. Zobrist 哈希 (ZobristHash)

hash = hash XOR table[row][col][color][state]，其中 state = revealed ? type + 1 : 0。

用于 TT 索引，与 positionKey 字符串哈希并行存在，服务不同场景。

5.6. 局面评估函数 (EnhancedEvaluator)

采用线性加权多特征模型，从当前方视角返回 score（正值 = 有利）：

5.6.1. 七维特征

维度	算法
子力 Material	明子固定值：将 10000、车 900、马 400、炮 450、兵 50、士象 200；过河兵/过河士象额外加成
暗子期望	每个未翻子按 virtualType 基准值取平均 × 数量 + 暗子奖励 +5/子
位置 PST	车/马/炮/兵 10×9 位置分表，中心与过河加权
机动性 Mobility	合法走法数 × 3
将帅安全 King Safety	士象护卫 +30/邻格；己方将越靠后 +10/行；被将军 -150
威胁 Threats	对方最便宜攻击子 vs 我方子价值 → MVV-LVA 式挂子惩罚
兵形 Pawn Structure	相邻兵 +10
残局猎杀 King Hunt	对方非王子力 < 1500 时激活：攻击子逼近对方将 + 压缩将活动空间 + 己将军 +80

5.6.2. 暗子子力处理 (Expectimax 思想的静态近似)

评估阶段不对暗子做蒙特卡洛，而用 virtualType 期望值：

```
darkValueSum += getBaseValue(virtualType)
material += (darkValueSum / darkCount) * darkCount
```

这是不完全信息下的一阶矩估计，计算代价 O(暗子数)，适合搜索内高频调用。

5.6.3. 概率偏置 (ProbabilityAgent)

搜索前计算:

```
evalBias = getExpectedValue(己方) - getExpectedValue(对手)
```

注入 quiescence 的 standPat 与叶子评估, 使搜索在暗子较多时偏向「暗子池期望值更高」的一方, 不直接选着。

5.7. Hard 档: Belief Sampling (BeliefAlphaBetaBot)

这是处理对手暗子不确定性的核心算法, 属于 Expectimax / 信念状态采样的工程实现。

5.7.1. 算法流程

Belief Sampling 主循环

```
输入: authoritativeBoard, color, timeBudget
1. publicView = createAiPublicView(color)
2. candidates = Top-K 启发式排序后的合法走法 (K ≤ 6)
3. 对每个候选走法 m:
   expectedScore = 0
   for s = 1 .. N (默认 N = 4):
     sampleBoard = BoardSampler.fromPublicView(publicView, color, rng)
     // 对手每个暗子, 从剩余子力池无放回随机分配 type
     makeMove(m) on sampleBoard
     score_s = AlphaBeta(sampleBoard, opp, perSampleBudget)
     expectedScore += -score_s // negamax 对手视角取反
   E[m] = expectedScore / usedSamples
4. return argmax_m E[m]
```

5.7.2. 剩余子力池约束 (BoardSampler)

初始池 = {2 车, 2 马, 2 炮, 5 兵, 2 士, 2 象} (不含将/帅)。减去对手已翻开明子的 type 计数, 对 hidden 暗子 shuffle 后逐一分配。保证采样与已公开信息一致——不会出现“场上已有 2 车再采样第 3 车”的逻辑错误。

5.7.3. 时间分配

```
perSample = max(30ms, remaining / remainingSlots)
budget < 3s → 降级为 2 采样 × 4 候选
```

每次采样独立 new OptimizedAlphaBeta(), 避免 TT 跨采样污染。

5.7.4. 理论定位

- 不是精确博弈树求解 (那需要 POMDP / 信息集搜索, 指数级复杂度)
- 是对对手 type 的蒙特卡洛确定化 + 完全信息 Alpha-Beta 的期望最大化
- 与 Medium 的本质区别: Medium 把对手暗子当 UNKNOWN 固定评估; Hard 对 type 分布多次采样取期望

5.8. 算法整体数据流

客户端 moveRequest 路径:



Listing 13: 走子处理数据流 — 规则引擎 + 终局 + 复盘记录
AI 选着路径 (Hard 档为例):



Listing 14: AI 选着全链路 — Belief Sampling → Alpha-Beta → 期望最大化 → 合法性兜底

5.9. AI 测试

用例	验证内容	结果
AiFairnessTest	AI 不透视对手暗子；三档输出均为合法走法	已实现
BoardUndoTest	搜索中 makeMove / undoMove 棋盘一致性	已实现
TranspositionTableTest	TT 命中率与正确性 (64-bit Zobrist 哈希碰撞验证)	已实现

5.10. 已知算法局限

局限	如实说明
Belief 采样数有限	默认 4×6 采样；对方 type 后验未建模（均匀剩余池采样，未考虑走法意图）
暗子评估用均值	virtualType 一阶矩估计，未做二阶不确定性惩罚（方差未建模）
Hard 独立 TT	每采样独立 new OptimizedAlphaBeta()，正确但重复计算多；默认 4 个 belief 采样 × 最多 6 个候选着法，实际搜索深度由局面分支数与时间预算决定，非固定 4-6 层
长捉判定简化	极端连环捉/隔子捉需人工复核
无残局库	残局依赖搜索到底 + EndgameAgent 加深（子力 ≤ 12 时 30s 预算）
AI 视角翻子	搜索中对手暗子翻开时用 virtualType 而非真实 type（防透视，但搜索树内翻子语义与真实对局略有偏差）

5.11. 三档 AI 代码对应表

难度	实现类	核心策略	时间	随机性	关键参数 (AiConfig)
Easy	EasyRuleBot	启发式排序 + Top-K 随机选择	≤ 500ms	高	topKRandom=8, timeLimitMs=min(500,humanBudget)
Medium	AlphaBetaBot	Agent 编排 + Alpha-Beta 搜索	≈ 5s	低	topKRandom=1, beliefSamples=0, 迭代加深 + TT + Aspiration ±80 + PVS + LMR
Hard	BeliefAlphaBetaBot	Belief Sampling + Alpha-Beta 期望搜索	≈ 5s	低	beliefSamples=4, maxCandidatesForBelief=6, topKRandom=1

5.12. Hard AI 选着流程

```
createAiPublicView (对手暗子脱敏)
↓
generateStrictLegalMoves (候选走法列表)
↓
取前 maxCandidatesForBelief(=6) 个候选着法
↓
对每个候选进行 beliefSamples(=4) 次采样
├ BoardSampler 从剩余子力池随机确定化
├ 每采样独立 new OptimizedAlphaBeta()
├ 执行 Alpha-Beta 迭代加深搜索
↓
对每候选的 4 次采样分数求平均 (期望值)
↓
选择期望分最高的走法
```

Listing 15: Hard AI Belief Sampling 选着流程 — 采样次数与候选数由 AiConfig 参数控制

源配置: `AiConfig.forLevel(HARD, humanBudgetMs)` → `beliefSamples=4`,
`maxCandidatesForBelief=6`, `topKRandom=1`。实际搜索深度不由参数固定, 而由迭代加深在时间预算内自动决定。

6. 客户端设计

Unveil 采用双端客户端策略: 控制台客户端 (`jieqi-client`) 面向课程验收与协议联调, Web 前端 (`jieqi-web`) 面向产品化演示与完整用户旅程。二者共用同一套 WebSocket JSON 协议 (端口 8887), 服务端 `WsGameServer` 为唯一规则权威。

6.1. 控制台客户端 (`jieqi-client`)

面向开发者、教师验收与组间互操作测试, 强调命令可脚本化与协议字段可读。

功能	说明
WsGameClient	WebSocket 客户端: <code>match</code> / <code>move</code> / <code>chat</code> / <code>replay</code> / <code>rematch</code> / <code>ai</code> / <code>watch</code> 等命令, 每条命令映射一条 JSON <code>messageType</code>
ConsoleUI	10×9 ASCII 棋盘渲染; ? 表示暗子; 红方大写 / 黑方小写; 行棋方高亮提示
Main 菜单	9 种模式: WS/TCP 服务器、WS/TCP 客户端、本地人机、AI 自动对弈、自检等统一入口

功能	说明
复盘命令	<code>replay</code> 进入复盘模式 → <code>n</code> (下一步) / <code>p</code> (上一步) / <code>0</code> (开局帧) / <code>end</code> (终局帧) / <code>g <n></code> (跳转第 <code>n</code> 帧) / <code>q</code> (退出)

典型验收路径：启动菜单 → 选 WS 客户端 → `login` → `match` → 双方 `ready` → `first true/false` 协商先手 → `move e6 e5` 走子 → 终局后 `replay` 逐步回看。

6.2. Web 前端 (jieqi-web)

技术栈：Vue 3 + TypeScript + Vite + Pinia，开发端口 5173。状态集中在 `stores/game.ts`，WebSocket 收发与协议解析与服务端字段一一对应。

页面/组件	说明	对应协议
LoginView	随机昵称（如「隐形小炮兵」）+ emoji 头像； <code>userId</code> 与昵称解耦	<code>Login</code> → <code>loginResult</code>
LobbyView	真人对战 / 人机三档 / AI 观战 / 房间对战四入口；二次确认防误触	<code>startMatch</code> / <code>startAiGame</code> / <code>createRoom</code>
GameView + ChessBoard	Canvas 10×9 棋盘；暗子 ? 显示；选中高亮 + 可走格提示；步时倒计时 65s	<code>move</code> → <code>moveResult</code> / <code>gameStart.initialBoard</code>
CapturedTray	被吃棋子信息差展示：我方吃子可见真实身份，被吃暗子隐藏身份	<code>moveResult.captured</code> + <code>visible</code>
操作面板	认输 / 手动加时 +30s / 提和；AI 模式额外提供暂停、结束	<code>resign</code> / <code>addTime</code> / <code>drawOffer</code> / <code>pauseGame</code>
局内聊天	快捷消息模板 + 15 种 emoji；≤ 120 字符文本	<code>chat</code> → <code>chatMessage</code>
终局弹窗	胜负 + 原因 + 暗子揭晓（上帝视角）；「查看棋局」「再来一局」「返回大厅」	<code>gameOver.capturedReveal</code>
复盘控件	开局 / 上一步 / 滑块 / 下一步 / 终局；3s 超时兜底	<code>replayRequest</code> → <code>replayFrame</code>

6.2.1. 前端状态流

Web 前端状态流覆盖从登录到终局复盘的全链路。前端代码位于 `jieqi-web/src/stores/game.ts`，其中 `getValidMoves`、`isInCheck`、`isCheckmate`、`isStalemate` 用于前端交互提示（可走点高亮、将军/终局提示），不作为权威规则判定依据——最终合法性以服务端 `moveResult.valid` 和 `gameOver` 为准。


```

Login → loginResult → 跳转 /lobby
↓
startMatch / startAiGame / createRoom → matchSuccess → 进入 /game
↓
gameStart → parseInitialBoard → displayBoard 渲染 (Canvas 10×9)
↓
用户点击棋子 → getValidMoves 生成可走点提示 → 点击目标格 → sendMove
↓
moveResult → applyMove (更新 displayBoard + flipResult 翻子动画)
├─ valid=true → 棋盘同步 + 切换回合
├─ valid=false → toast 提示 + 棋盘不变
↓
gameOver → capturedReveal 弹窗 (被吃暗子身份揭晓)
├─ 「查看棋局」→ enterReplayMode → replayBoard 切换
│   └─ sendReplayRequest(stepIndex) → replayFrame → replayBoard 渲染
│       └─ prev/next/跳至 → 同流程
├─ 「再来一局」→ rematchRequest → 新 gameStart
└─ 「返回大厅」→ /lobby

```

6.2.2. Web 界面展示

以下按用户实际路径组织截图说明：每一屏标注「用户看到什么 → 点了什么 → 服务端返回什么」，便于答辩时对照演示。

6.2.2.1. 登录与大厅

场景 S1 - S2：进入系统并选择对战模式

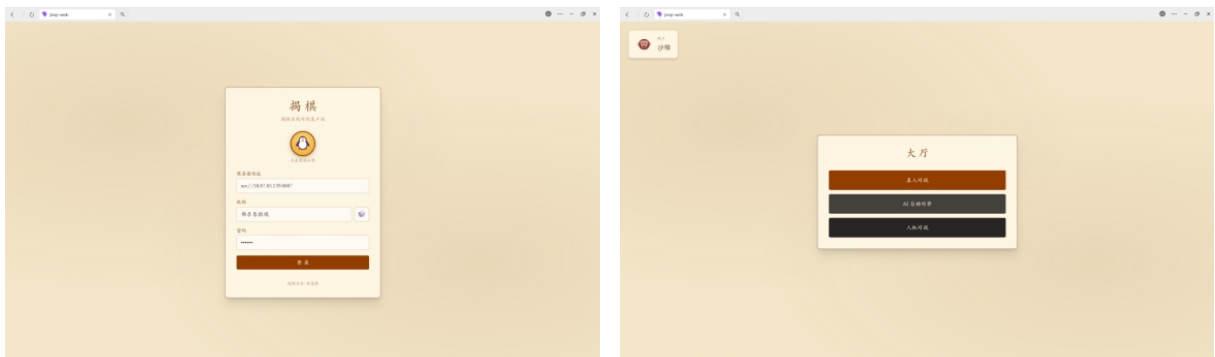


Figure 5: 登录页（左）与大厅主菜单（右）

界面区域	用户操作	系统响应
登录页 • 头像区	点击头像循环切换预设表情（共 16 种）	本地 nextAvatar(), 无需请求
登录页 • 昵称	输入昵称或点击“随机生成”按钮（如「佛系象棋魂」「隐形小炮兵」）	本地 randomNickname(); userId 独立生成 u_xxx 避免重名
登录页 • 服务器地址	填写 ws://host:8887（默认课程公共端口）	建立 WebSocket 长连接；底部显示「已连接 / 未连接」

界面区域	用户操作	系统响应
登录页 • 登录按钮	点击「登录」	发送 login JSON → 收到 loginResult (success + userId + nickname + avatar) → 跳转 /lobby
大厅 • 三模式入口	「真人对战」→ 匹配子面板； 「AI 自动对弈」→ 观战确认； 「人机对战」→ 难度选择	分别触发 startMatch / startAiBattle / startAiGame
大厅 • 用户信息卡	左上角显示当前 emoji 头像 + 昵称	来自 loginResult, 全程 Pinia 持久化

场景 S3 - S4：匹配与房间



Figure 6：随机匹配中（左）与创建/加入房间（右）

界面状态	用户操作	服务端行为
匹配中	点击「随机匹配」后按钮变为「匹配中…」不可重复点击	RoomManager 入队；匹配成功广播 matchSuccess (roomId + opponentId + nickname)
房间对战 • 创建	点击「创建房间」	createRoom → 返回 6 位房间号；状态栏提示「把房间号告诉对方」
房间对战 • 加入	输入 6 位房间号 → 「加入房间」	joinRoom → roomInfo 广播双方对手信息
返回	「返回」回到主菜单	取消匹配 / 离开房间（若已在房间）

场景 S5：双方准备就绪



Figure 7: 房间创建等待（左）与双方准备就绪双端对比（右）

界面要素	说明	协议节点
房间信息条	显示「房间：167302 对手：xxx」	roomInfo 推送 opponentId / nickname
等待加入	房主侧：创建后输入框置灰，提示等待对方	对手 joinRoom 后信息条更新
我已准备	双方均点击后高亮为不可重复态	双方 Ready → 服务器 requestFirstHand 协商 → gameStart
双端对比	左：红方视角；右：黑方视角；对手昵称互为镜像	同一 roomId, yourColor 不同

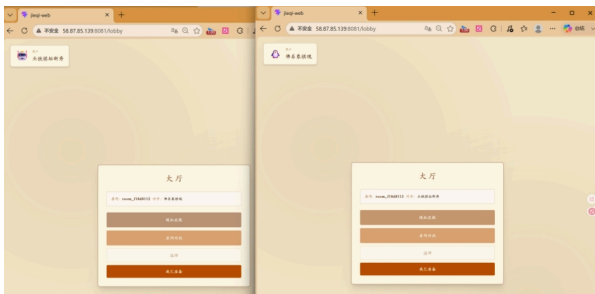


Figure 8: 真人对战双端准备界面 — 双方均已点击「我已准备」，等待服务器下发 gameStart

6.2.2.2. AI 对弈模式

场景 S6 - S7：人机对战与 AI 自动对弈观战



Figure 9: 人机难度选择（左）与 AI 自动对弈确认弹窗（右）

模式	界面说明	后端调度
人机对战 • 入门	Easy: 启发式 + 30% 随机, 响应 < 500ms	startAiGame + aiLevel: easy → EasyRuleBot
人机对战 • 标准	Medium: Alpha-Beta + Agent 编排, 5s 搜索	aiLevel: medium → AlphaBetaBot / JieqiAgent
人机对战 • 挑战	Hard: Belief Sampling 期望最大化, 5s 多采样	aiLevel: hard → BeliefAlphaBetaBot
AI 自动对弈	确认弹窗: 「观战两位 AI 自动对弈, 全程可暂停/结束」	startAiBattle → 服务端双 Bot 循环走子, 客户端仅接收广播
二次确认	所有 AI 模式均有「确认 / 取消」防误触	取消则不发送 start 消息, 留在 lobby



Figure 10: 人机对弈进行中（左：标准 AI）与 AI 自动对弈观战（右：双 AI 互博）

界面区域	人机对战	AI 自动对弈观战
左侧信息卡	显示 AI 难度名称 + 步时倒计时	显示「AI 自动对弈」+ 双方 AI 昵称
中央棋盘	用户执红（默认），点击选子 → 点击目标格走子	只读棋盘，自动播放双方 moveResult
右侧操作	认输 / 加时 / 提和（真人对战同款）	「暂停对局」「结束对局」— 对应 pauseGame / 强制终局
被吃子区	CapturedTray 信息差渲染	同左，终局后揭晓
状态栏	房间号 + 「已连接 / 已结束」	房间号 + 观战模式标识

6.2.2.3. 真人对战界面

场景 S8：开局与行棋

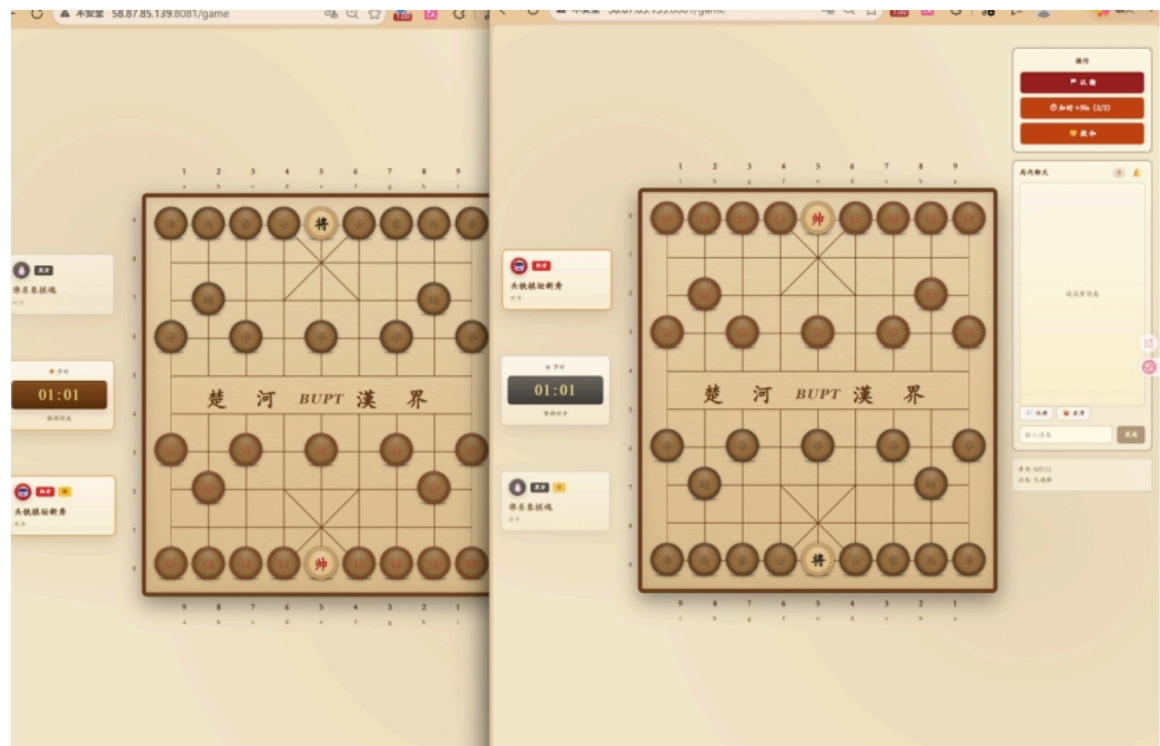


Figure 11: 真人对战开局双端对比 — 红方「轮到你走」/ 黑方「等待对手」

界面要素	红方视角（左）	黑方视角（右）
棋盘	10×9 Canvas；楚河汉界含 BUCT 标识；坐标 a - i / 0 - 9	镜像布局，己方始终在下方
步时倒计时	「01:01 轮到你走」— 65s 步时	「01:01 等待对手」— 同步服务器 turnStartTime
操作面板	认输 / 加时+30s（2/2）/ 提和	同左；提和需对方 drawAccept 才终局
聊天区	「还没有消息」→ 可发快捷语 / emoji / 自定义文本	双方 chatMessage 广播同步
走子交互	选中棋子 → 高亮可走格 → 点击目标 → 发送 move	收到 moveResult 后棋盘同步 + flipResult 翻子动画
非法走子	moveResult.valid=false，棋盘不变，toast 提示	服务端 RuleValidator 权威拒绝

6.2.2.4. 局内操作与聊天

场景 S9：辅助功能与社交互动



Figure 12: 操作面板（左）与认输确认弹窗（右）

功能	交互设计	服务端处理
认输	二次确认：「认输后对局会立即结束，对方获胜」	<code>resign</code> → <code>gameOver</code> (<code>reason=resign</code> , <code>winner=对方</code>)
加时 +30s	每步最多 2 次；按钮显示剩余次数 (2/2)	<code>addTime</code> → <code>timeBonus</code> (<code>turnStartTime</code> 延后 60s)
提和	发送提和请求，等待对方响应	<code>drawOffer</code> → 对方 <code>drawAccept</code> 则和棋 / <code>drawDecline</code> 则继续
暂停（AI 模式）	暂停后 AI 停止走子，显示「继续对局」	<code>pauseGame</code> / <code>resumeGame</code> → <code>gamePaused</code> / <code>gameResumed</code>



Figure 13: 局内聊天（左）与表情选择器（右）

聊天能力	说明
快捷消息	预设模板一键发送（如「要不要我帮你走这步?」「稳住，我们能输！」）
表情面板	3×5 共 15 种预设表情；点击即发送
自定义输入	底部输入框 + 「发送」；单条 ≤ 120 字符（与服务端校验一致）
消息归属	显示「黑方 你」/「红方 你」+ 时间戳；仅本局参与者可见
提示音	新消息可选提示音（chatSoundOn 开关，默认开启）

6.2.2.5. 聊天室协议与实现

协议定义：

协议层	消息类型	说明
WebSocket/JSON（主）	chat (C→S) / chatMessage (S→C)	客户端发送 chat， 服务器广播 chatMessage
TCP 文本帧（附录 B）	MSG_CHAT = 10	格式：playerColor playerName message

关键文件：JsonMessageTypes.java（CHAT / CHAT_MESSAGE 常量）、JsonMessages.java（构建带服务器时间戳的 chatMessage JSON）、Protocol.java（TCP MSG_CHAT=10 + buildChatMsg()）。

服务端实现（WsGameServer.handleChat()，行 493-518）：

- 校验链：用户已登录 → 已加入房间 → 游戏已开始且处于 PLAYING 状态
- AI 对局拦截：若 room.hasAiOpponent() 或 room.isAiBattle()，返回错误“仅真人对局支持聊天”
- 内容净化（sanitizeChatContent()）：替换换行符为空格，去除首尾空白，截断至 120 字符
- 使用服务器时间戳（不信任客户端时间）
- 通过 broadcastRoom() 向双方及观战者广播

TCP 路径（ClientHandler.sanitizeChatPayload()，行 146-157）：

- 频率限制：两次发言间隔 ≥ 10 秒（CHAT_MIN_INTERVAL_MS = 10_000L），超频返回提示
- 长度限制：200 字符
- 广播至对局双方

两协议差异：

特性	WebSocket (8887)	TCP (8888)
字符限制	120	200

特性	WebSocket (8887)	TCP (8888)
频率限制	无	10 秒/条

前端消息流：

```

客户端 A → sendChat() → WS JSON: {messageType: "chat", content: "..."}
→ WsGameServer.handleChat() 校验净化 → broadcastRoom()
→ 客户端 A + B 收到 {messageType: "chatMessage", fromUserId,
fromColor, content, timestamp}
→ GameView.vue 渲染聊天气泡 (mine = fromUserId === this.userId)
```

Listing 16: 聊天消息全链路

设计要点：仅真人可用（服务端 + 前端双重校验）；服务器时间戳（遵循“客户端时间戳不被信任”原则）；无持久化（消息仅内存传输，广播后即丢弃，客户端上限 60 条）；内容净化（服务端替换换行、截断超长内容）；课程扩展功能（INTERFACE. typ 标记为“可选扩展” MSG 10，非核心协议要求）。

6.2.2.6. 终局结果

场景 S10 - S11：终局弹窗与后续操作

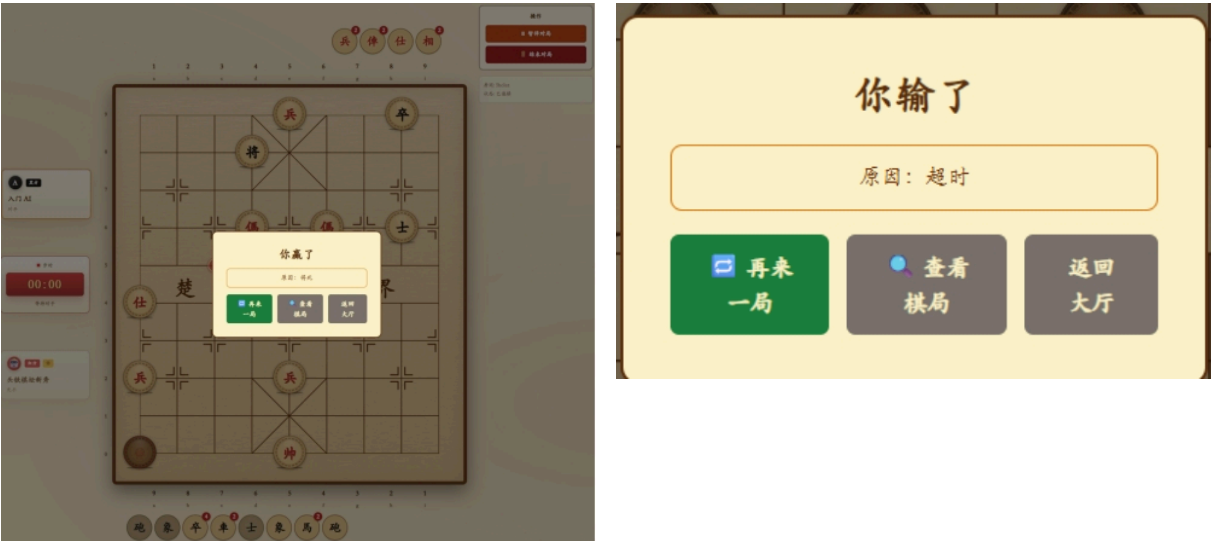


Figure 14: 终局弹窗 — 将死获胜（左）与超时失败（右）

终局类型	界面展示	后续路径
将死获胜	「你赢了」+ 原因：将死；被吃暗子以真实身份展示在棋盘外	「再来一局」→ rematchRequest; 「查看棋局」→ 复盘; 「返回大厅」→ / lobby
超时失败	「你输了」+ 原因：超时; loserId 由服务器判定	同上三条路径

终局类型	界面展示	后续路径
和棋 / 认输	reason 字段对应 draw_agreement / resign 等	capturedReveal 上帝视角揭晓全部被吃暗子身份
状态同步	右侧状态栏变为「已结束」； 步时归零 00:00	gameOver 广播后棋谱落盘 + replay.json 持久化

场景 S12: 复盘 (详见 § 7 棋谱与复盘)

终局后点击「查看棋局」→ 进入 replayMode → 底部出现步进控件 → 每步发送 replayRequest(stepIndex) → 渲染 replayFrame.board 快照。对局中暗子复盘按玩家视角脱敏；终局后按快照上帝视角展示。

已知边界: Web 端逐步复盘依赖终局后房间仍保留在服务端内存中 (wsGameServer 未重启、未超时清理); 历史 replay.json 文件浏览尚未实现 (文件已落盘于 records/, 但无 HTTP 接口或前端页面加载历史对局)。

6.2.3. 复盘模式架构



Listing 17: Web 前端复盘数据流 (后端是数据源, 前端是播放器)

6.3. Web 端当前进展与已知限制

6.3.1. 已完成 (可演示)

模块	状态与证据
三页面路由	已实现 完成: LoginView / LobbyView / GameView
棋盘渲染	已实现 完成: ChessBoard.vue Canvas + 暗子 ? 标记 + 选中高亮
信息差展示	已实现 完成: CapturedTray 被吃区脱敏渲染

模块	状态与证据
四种对战模式	已实现 完成：真人匹配 / 房间对战 / 人机三档 / AI 自动观战
局内交互	已实现 完成：聊天、emoji、提和、认输、手动加时、AI 暂停
终局闭环	已实现 完成：胜负弹窗 + capturedReveal 上帝视角 + rematch 三态 UI
复盘播放器	已实现 完成：replayRequest/replayFrame + 步进控件 + 3s 超时兜底
截图素材	已实现 完成：18 张语义化命名并嵌入本报告 § 6.2.1
控制台双轨	已实现 完成：jieshi-client 命令行同协议链路验收

6.3.2. 待强化 / 已知限制

项	状态	说明
复盘生产部署	部分实现 依赖后端版本	UI 就绪；需确保 <code>WsGameServer.handleReplayRequest</code> 随最新 Fat JAR 部署；Web 端复盘依赖终局后房间仍在服务端内存中，历史 <code>replay.json</code> 文件浏览尚未实现
走子错误码细化	部分实现 P2	前端 toast 多为通用文案，未细分到具体规则分支
Docker 一键 Web	部分实现 实验性	后端 Docker 已就绪；前端需 <code>npm run dev</code> 单独起，未整合
商业化 polish	未实现 非目标	无 3D 棋盘、无移动端适配、无排行榜（与产品定位一致）

Web 前端在产品矩阵中的定位：控制台「完整」+ Web「完整（复盘依赖服务端部署）」——已从 P2「规划中 GUI」升级为答辩主展示入口。

7. 棋谱与复盘

7.1. 三类产物

产物	路径	说明
文字棋谱	records/<id>.jieqi	走法文本，含回合号 + 红黑标识 + 翻子标记，供阅读导出
复盘 JSON	records/<id>.replay.json	逐步 Board 快照（含暗子真实类型），防御性拷贝 + 终局落盘
内存时间线	Game.replayTimeline	List<ReplayFrame>，对局进行中实时支持 replayRequest 查询

7.2. 帧编号模型

```
stepIndex=0    开局帧（无 move，仅初始棋盘快照）
stepIndex=1    第 1 手后的完整棋盘
stepIndex=2    第 2 手后的完整棋盘
...
stepIndex=N    终局后最后一帧（含 gameOver 状态）
```

Listing 18: 复盘帧编号（0-indexed）

每帧含：stepIndex、move（可 null）、boardSnapshot（防御性拷贝）、currentTurn、status、timestamp、captured。

7.3. 产生时机

```
gameStart → Game.recordReplayInitialIfNeeded()    # stepIndex=0
|
| (每步 loop)
processMove → replayTimeline.recordAfterMove()    # stepIndex=1,2,...
|
gameOver → persistReplay(game)                    # JSON 落盘
```

Listing 19: 复盘数据产生全流程

7.4. 复盘协议

messageType	方向	关键字段
replayRequest	C→S	stepIndex（可选，缺省返回最后一帧）
replayFrame	S→C	roomId, stepIndex, totalSteps, currentTurn, status, board[], move?, captured?

board 数组每元素：{ x: "a"-"i", y: 0-9, color: "red"|"black", piece: "rook"|..., visible: true|false }。

7.5. 权限与视角

场景	显示
对局中复盘	按玩家视角：对手暗子不暴露真实 type
终局后复盘	上帝视角：快照含完整 Board（capturedReveal 揭晓所有暗子身份）
非房间成员	拒绝：仅本局参与者可 replayRequest

7.6. 前端复盘引擎

复盘为纯客户端交互式对局回放系统，完整实现在 jieqi-web/src/stores/game.ts（Pinia Store）。无独立服务端 API 端点，无 AI 参与复盘分析。

7.6.1. 核心数据模型

前端以 BoardState 树 组织快照，支持主链回放与分支探索：

字段	含义
id: string	唯一快照标识（s1, s2, ...）
board: Piece[]	完整深拷贝棋盘快照
parent: string null	父状态 ID（null = 根节点）
mainNext: string null	主链下一状态 ID
branches: string[]	分支状态 ID 列表（假想变化）
moveRecord?: MoveRecord	产生此状态的走法（before / after / player / timestamp）

形成树形结构：一条线性「主链」(mainNext) 代表实际对局过程，分支 (branches[]) 代表假想变化。

7.6.2. 完整流程

阶段一：对局中录制 — 每步 applyMove() 后调用 recordBoardSnapshot(): 深拷贝当前棋盘 → 创建 BoardState 节点 → mainNext 链接父节点 → 更新 currentStateId。开局初始棋盘在 gameStart 时快照。

阶段二：进入复盘 — 对局结束显示结算弹窗 → 「查看棋局」→ 「进入复盘」→ enterReplay(): 设置 isReplayMode = true, 导航到最新主链状态。

阶段三：导航操作 — goNext() (沿 mainNext 前进) / goPrev() (沿 parent 后退) / goToStart() / goToEnd(), 每次交换回合。

阶段四：自由分支 — 复盘模式下点击棋子触发 handleReplayCellClick(): 校验回合归属 → 选择目标 → executeReplayMove() 创建分支 BoardState。翻子时通过 findRealTypeInMainChain() 沿主链查找真实类型 (非凭空生成)。将帅被吃后设置 branchKingCaptured = true, 锁定分支。

阶段五：退出 — exitReplay() 恢复最新主链状态，清除分支状态，返回结算视图。

7.6.3. 服务端支持

复盘功能无独立 API 端点。相关服务端消息仅 gameStart (提供 initialBoard 种子)、moveResult (触发快照录制)、gameOver (标记对局结束并启用复盘 UI)。服务端通过 GameRecordStore.save() 将棋谱持久化至 records/{gameId}.jieqi, 前端复盘不读取这些文件，完全依赖内存 boardStates[]。

7.6.4. 架构总结

[Server]	[Frontend Browser]
Game.java	game.ts (Pinia)
+ Board.moveHistory[]	+ boardStates[] (树)
+ GameRecord (text)	+ isReplayMode
+ GameRecordStore.save()	+ enterReplay()/exitReplay()
`- records/*.jieqi	+ goPrev()/goNext()
	+ executeReplayMove() (分支)
WsGameServer:	+ findRealTypeInMainChain()
`- broadcastGameOver()	+ recordBoardSnapshot()
`- persistRecord(game)	

Listing 20: 复盘系统架构 — 服务端持久化与前端快照树解耦

7.6.5. 设计要点

1. 纯客户端实现：复盘所有交互逻辑在前端完成，无需服务端复盘 API
2. 树形快照结构：BoardState 树支持主链回放 + 任意分支探索
3. 主链暗子查询：分支翻子沿主链向前查找真实类型，而非凭空生成
4. 将帅保护：分支中将帅被吃后自动锁定，防止无意义走法
5. 无 AI 参与：jieqi-ai 模块专用于对局走子选择，不参与复盘分析、走法点评或自动复盘
6. 棋谱文件独立：服务端持久化 .jieqi 文件用于存档，与前端复盘系统解耦

8. 测试与质量

测试策略遵循“接口先行、测试驱动”原则：核心领域逻辑（jieqi-core）以单元测试覆盖全部规则分支，协议集成（jieqi-server）以端到端测试验证完整消息链路，AI 模块（jieqi-ai）以正确性断言确保搜索合法性与公平性。所有测试仅使用 JUnit Jupiter 5.11.4 内置断言，未引入 Mockito、AssertJ 等第三方测试框架。测试执行通过 Maven Surefire 插件，CI 配置于 `.github/workflows/ci.yml`（push/PR 自动触发）。



Figure 15: 测试与质量体系总览 — 142 项分布 • CI 流水线 • 三层测试策略

8.1. 测试汇总

指标	数值
自动化用例总数	142
通过 / 失败 / 跳过	142 / 0 / 0
自检脚本	verify.ps1 → OK: verify passed
代码规模	63 主代码文件 (8 500 行) + 51 测试文件 → 合计 114 文件 (10 500 行)

所有 142 个用例覆盖 jieqi-core (89)、jieqi-ai (16)、jieqi-server (37) 三个模块，0 失败 0 跳过。verify.ps1 脚本镜像 CI 流水线步骤，用于 push 前本地一键自检。

8.2. 分模块结果

模块	测试类	用例	覆盖重点
jieqi-core	28	89	七种走法、暗子规则、送将/照面、将死/困毙、40 步和、长将长捉、棋谱、复盘时间线
jieqi-ai	9	16	AB 搜索、三档 Bot 合法性与公平性、置换表、Agent 编排
jieqi-server	12	37	TCP 集成 7 + WS 集成 21 + 复盘落盘 2 + 记录存储 7
jieqi-client	0	—	由 WS 集成测试间接覆盖
jieqi-app	0	—	Fat JAR 启动器，无单测
合计	49	142	全通过

8.3. 关键测试场景（摘录）

以下 9 个场景摘录自 142 项测试中最具代表性的用例，覆盖规则边界（R 系列）、AI 正确性（A 系列）、网络协议（N 系列）与复盘/观战（P 系列）四条主线。每个场景均由对应测试类直接验证，构建于 `verify.ps1` 自动化流程中。

编号	场景	测试方法	结果
R11 - R13	送将 / 照面 / 解将	RuleEdgeCaseTest 11 项	已实现
E01 - E03	将死 / 困毙 / 40 步和	EndgameJudgeTest	已实现
A03	AI 不透视对手暗子	AiFairnessTest	已实现
A06	搜索 undo 棋盘一致性	BoardUndoTest	已实现
N01	WS 匹配 → Ready → gameStart	WsGameServerIntegrationTest	已实现
N04	非法走法拒绝	illegalMoveRejected	已实现
N05	步时超时判负	turnTimeoutEndsGame	已实现
P04	复盘帧 replayRequest	replayRequestReturnsFramesAfterResign	已实现
P05	观战 watch	watchJoinsActiveGameAsObserver	已实现

8.4. AI 性能观测

AI 模块的性能验证通过 `PerformanceTest.main()` 独立基准（非 JUnit 测试）在不同时间限制（1s/2s/5s/10s）下测量搜索深度与节点吞吐。三档 AI 的实际表现如下：

等级	典型步时	超时
Easy	< 500ms	0
Medium	1 - 3 秒（深度 8 - 12）	0
Hard	2 - 5 秒（Belief 多采样）	0（复杂中局可能 fallback）

Easy 档通过时间上限截断（`timeLimitMs=min(500, humanBudget)`）保证毫秒级响应；Medium 与 Hard 共享 5s 时间预算，但 Hard 因需要为每候选执行 4 次完整 AB 搜索，实际搜索深度低于 Medium，棋力通过采样期望值的统计稳定性补偿。

8.5. 需求与测试映射矩阵

验收点	主要风险	测试方式
七种棋子走法	走法边界错误（蹩腿/炮架/过河）	<code>BoardMakeMoveTest</code> • <code>RuleValidatorTest</code>
暗子机制	<code>type/virtualType</code> 混淆；暗子走法越界	<code>DarkPieceRuleTest</code> • <code>BoardAiPublicViewTest</code>
强化士象	暗士出九宫、明士禁足；暗象过河、明象限边	<code>DarkPieceRuleTest</code> 强化规则子项
送将与照面	非法局面未被拦截（将帅对面）	<code>RuleEdgeCaseTest</code> 11 项
终局判定	将死/困毙/超时判定错误；无吃子和计数遗漏	<code>EndgameJudgeTest</code> • <code>RuleEdgeCaseTest</code>
走子回滚	<code>makeMove/unmakeMove</code> 状态未完全恢复	<code>BoardUndoTest</code>
AI 公平性	AI 透视对手暗子 <code>identity</code>	<code>AiFairnessTest</code> • <code>BoardAiPublicViewTest</code>

验收点	主要风险	测试方式
AI 搜索一致性	搜索前后棋盘不一致 (TT 污染/undo 不完整)	OptimizedAlphaBetaRepetitionTest
复盘帧完整性	帧编号错误; 快照不独立 (引用污染); 竞态落盘空帧	ReplayTimelineTest + 服务端集成 2 项
WebSocket 状态同步	房间状态不同步; 消息丢失	WsGameServer 集成测 37 项
非法走法拒绝	服务端未拦截非法走子	非法走法拒绝测
重复局面判定	长将/长捉误判; 计数清零时机错误	EndgameJudge 长将/长捉子项

8.6. 详细测试清单

以下按模块列出关键测试类及其覆盖范围, 补充模块汇总表 (§ 8.2) 中未展开的细节。

8.6.1. jieqi-core 测试清单 (15 类)

领域规则、协议序列化、棋谱记录三大方向覆盖 jieqi-core 的全部核心逻辑。

领域规则测试 (8 类):

测试类	覆盖内容
CoordinateTest	坐标格式 "a0".."i9" 校验与行列转换
ChessPieceCoordTest	棋子坐标映射 (a0=红方左下, i9=黑方右上)
DarkPieceRuleTest	暗子规则: 士不能出宫、翻子后可过河、翻子动作不可吃子、生成动作不含送将
KingCapturedRuleTest	翻子吃将立即获胜、禁止翻自己的将
BoardExpectedValueTest	开局暗子期望值正数且对称
BoardSyncTest	BOARD_STATE 序列化/反序列化完整性
EndgameJudgeTest	将死、困毙、吃将终局、长将判负 (6 次)、兵长捉和棋 (6 次)、无捉无将不判负
GameEndgameTest	超时判负

协议层测试覆盖 TCP 帧解码健壮性、完整消息往返、JSON 序列化一致性:

协议与序列化测试（5 类）：

测试类	覆盖内容
FrameDecoderTest	单帧解码、粘包解码、半包处理、非法长度拒绝、超大负载拒绝、缓冲区溢出拒绝
ProtocolFrameIntegrationTest	登录帧往返、棋盘状态帧完整性
ProtocolMoveSerializationTest	走法序列化/反序列化（含翻子标记）、仅翻子往返
BoardJsonMapperTest	初始棋盘含将和暗子、棋子名称往返、过河棋子颜色正确
PieceJsonMapperTest	JSON 名称匹配教学规范、解析所有类型值、全部类型往返、颜色映射

棋谱测试（3 类）：

测试类	覆盖内容
GameRecordTest	标准记法导出、走法格式匹配接口规范
GameRecordImportTest	解析编号行并跳过注释、导出/导入往返
MoveNotationParseTest	翻子记法解析、普通走法记法解析

8.7. jieqi-server 测试清单（12 类）

服务端测试以集成测试为主，覆盖 TCP 与 WebSocket 双协议栈的完整消息链路。

`AbstractGameServerIntegrationTest` 为 TCP 测试提供 `loginAndDrain()`、`connectTwoPlayers()`、`awaitBoardState()`、`awaitMoveBroadcast()` 等阻塞断言基元，管理 `GameServer` 生命周期（随机端口避免冲突）。

TCP 集成测试（8 类，均继承基类）：

测试类	测试场景
GameServerLoginIntegrationTest	双人登录后收到 <code>GAME_START</code>
GameServerMoveIntegrationTest	红方走子后黑方收到 <code>MOVE</code> 广播
GameServerIllegalMoveIntegrationTest	错手走子返回 <code>ERROR 101</code>
GameServerDrawIntegrationTest	求和/同意求和广播 <code>GAME_OVER</code> (<code>AGREED_DRAW</code>)
GameServerResignIntegrationTest	认输广播 <code>GAME_OVER</code> 并持久化棋谱文件
GameServerTurnChangeIntegrationTest	走子后 <code>TURN_CHANGE</code> 广播双方
GameServerUnknownMsgIntegrationTest	未知 <code>msgType</code> 被忽略，后续走法正常处理

测试类	测试场景
GameServerChatIntegrationTest	聊天广播与频率限制验证

WebSocket 集成测试（1 类，21 方法）：

WsGameServerIntegrationTest 是项目中最全面的测试文件，使用内建 TestWsClient 实现端到端测试：登录、匹配、准备、游戏开始、完整对局流程、非法走子返回错误 2001、非己方回合返回错误 2002、聊天广播、认输、同意求和、拒绝求和、创建房间+加入码、AI 对局、AI 对局聊天拒绝、AI 对战观战、超时、断线终局、先手请求交换、未知消息类型返回错误 4001、未登录匹配返回错误 1001、取消匹配。

其他服务测试：

测试类	测试内容
MatchmakingServiceTest	单人自动匹配、房间不存在返回 NOT_FOUND、满房返回 ALREADY_STARTED
GameRecordStoreTest	棋谱文件保存/加载往返（使用 @TempDir）

8.8. jieqi-ai 测试清单（5 类）

测试类	测试内容
JieqiAgentTest	AI 在 3 秒内选择合法走法
EnhancedEvaluatorTest	评估函数在红黑视角间反对称
OptimizedAlphaBetaTacticalTest	搜索优先解决大子威胁（战术意识）
AgentOrchestratorTest	编排器使用第一个返回走法的子 Agent
ProbabilityAgentTest	暗子存在时概率 Agent 设置期望值偏置
EndgameAgentTest	盘面棋子多时终局 Agent 不激活

8.9. AI 性能基准

PerformanceTest.main()（main 源码，非 JUnit 测试）在不同时间限制（1s/2s/5s/10s）下基准测试搜索性能，输出深度/节点数/评分。

8.10. 测试框架与 CI

测试框架选型以简洁可复现为原则：仅依赖 JUnit Jupiter 5.11.4 内置断言，不引入额外测试库，降低答辩机环境依赖。Maven Surefire 插件在 mvn test 阶段自动发现并执行所有 *Test.java 文件，无需额外配置。

组件	配置
JUnit Jupiter	5.11.4 — 所有 Java 测试模块统一框架
maven-surefire-plugin	3.5.2 — 父 POM 测试执行引擎
GitHub Actions CI	.github/workflows/ci.yml: push/PR 到 main 触发, JDK 21 (Temurin), Maven 缓存, 步骤 mvn test -pl jieqi-core,jieqi-server,jieqi-ai → mvn compile → mvn package -pl jieqi-app -am -DskipTests
本地验证	scripts/verify.ps1 镜像 CI 步骤, push 前本地验证

未使用 Mockito、Spring Test、JaCoCo（代码覆盖率）、AssertJ、Hamcrest。所有测试仅使用 JUnit Jupiter 内置断言。这一策略的代价是 AI 子 Agent 测试使用匿名类桩代码、缺少覆盖率量化、无法区分单元/集成测试分组，但对课程验收而言，142 项全通过的确定性远高于工具链的完备性。

8.11. 测试质量评估

综合审视当前测试体系，其优势与不足如下：

优势：

- 协议驱动集成测试：WsGameServerIntegrationTest（21 方法）覆盖完整 JSON 消息协议端到端流程
- 游戏规则覆盖充分：EndgameJudgeTest（6 项）覆盖将死、困毙、吃将、长将、长捉等终局条件；DarkPieceRuleTest（6 项）覆盖暗子移动约束
- 协议序列化健壮：FrameDecoderTest（6 项）覆盖粘包、半包、非法长度、缓冲区溢出等边界
- CI 已配置且正常运行

不足：

- jieqi-client、jieqi-app、jieqi-web 三模块零测试覆盖
- 无 Mock 框架（Mockito 未引入），AI 子 Agent 测试用匿名类桩代码，不可扩展
- 无代码覆盖率工具（JaCoCo 完全缺失）
- AI 测试薄弱：仅 5 个基础测试，无特定搜索深度、非平凡局面评估正确性或性能断言
- 无参数化测试（手动循环代替 @ParameterizedTest）
- 无测试标签/分组（所有测试一同运行，未区分单元测试与集成测试）

8.12. 已知未修

优先级	项	说明
P2	走子错误原因码	RuleValidator 仅返回 boolean, 不细分“哪个规则违反”
P2	长捉复杂分类	极端连环捉、隔子捉需人工复核
P3	jieqi-client 单测	依赖 WS 集成间接覆盖
P3	jieqi-web E2E	Vue 前端无 Playwright 自动化

9. 部署与运行

本章说明 Unveil 在本地开发机、答辩演示机与 Docker 容器三种场景下的构建、启动与验收方法。部署原则（秦博宇 • 服务端交付）：WsGameServer 为唯一规则权威；客户端/Web 连 `ws://host:8887`；扩展消息（replay / rematch / addTime）须用最新 Fat JAR，避免 `~/.m2` 过期 SNAPSHOT。

9.1. 部署拓扑

Unveil 采用 Fat JAR 单中枢 部署模型：jieqi-app 模块通过 Maven Assembly 将所有依赖（core/server/ai）打包为单个 `unveil-jieqi.jar`，启动时只需 JDK 21 运行时，无需外置应用服务器或数据库。服务端以 WsGameServer（端口 8887）为统一入口，所有客户端（控制台/Web/双 AI 对弈）通过同一 WebSocket 地址互联。部署场景覆盖三种典型环境：

- 本地开发机：JDK 21 + Maven + Node.js 全栈，适合编码调试
- 答辩演示机：仅需 JDK 21（或 Docker），通过 `verify.ps1` → `dev-server.ps1` → `demo.ps1` 三条脚本完成全流程
- Docker 容器：无 JDK 宿主机一键启动 WS 服务，牺牲 TCP 8888 与 Web 前端集成换取零依赖

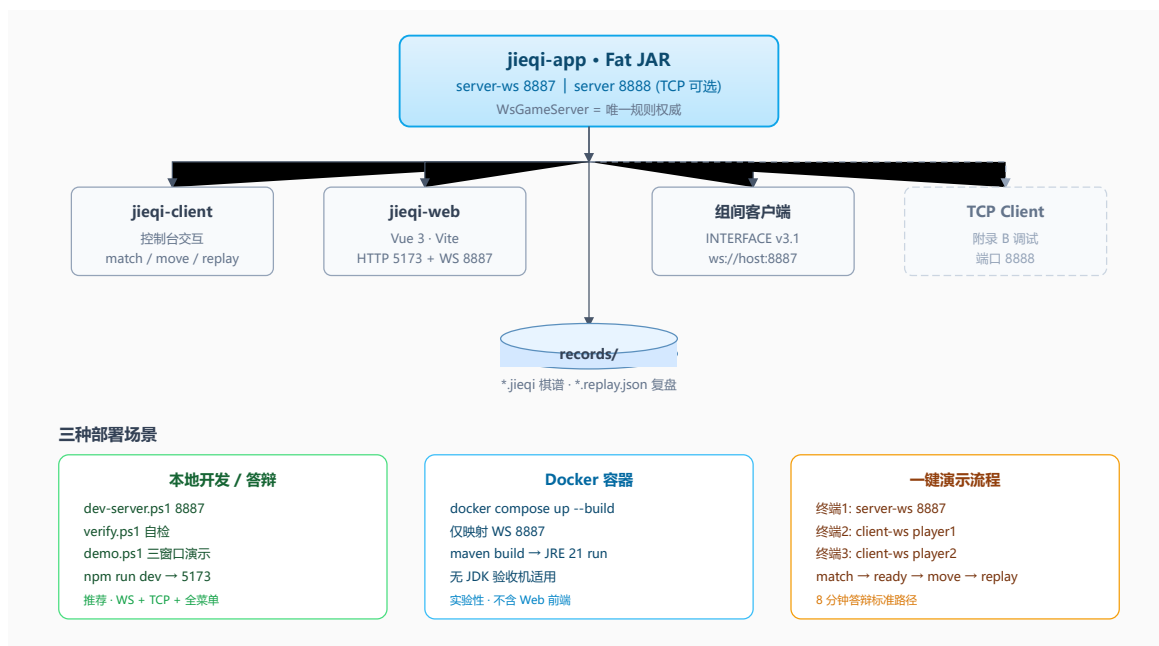


Figure 16: 部署拓扑 — Fat JAR 中枢 • 四客户端 • records 落盘 • 三种运行场景
各组件端口与职责如下：

组件	默认端口	协议	用途
WsGameServer	8887	WebSocket + JSON v3.0	课程主通道 • 组间互操作 • Web/控制台
GameServer (TCP)	8888	文本帧 附录 B	legacy 调试; Docker 默认不映射
jieqi-web (Vite)	5173	HTTP + WS 客户端	浏览器 GUI; 需 Node.js 18+
records/	—	文件系统	棋谱 *.jieqi + 复盘 *.replay.json

9.2. 环境要求

9.2.1. 必装组件

组件	版本	检查命令	期望
JDK	21. x	java -version	major version = 21
Maven	3.9+	mvn -version	Java version: 21
Git	2. x+	git --version	—
PowerShell	5.1+ / 7+	\$PSVersionTable.PSVersion	自检/演示脚本

9.2.2. 可选组件

组件	版本	用途	检查
Node.js	18+	jieqi-web 前端	<code>node -v</code>
npm	9+	Vue 依赖安装	<code>npm -v</code>
Docker Engine	24+	无 JDK 一键起 WS	<code>docker --version</code>
Typst	0.14+	编译 INTERFACE / 文档 PDF	<code>typst --version</code>

9.2.3. JDK 环境核对清单（答辩机必做）

检查项	命令	通过标准
Java 运行时	<code>java -version</code>	输出含 21.x
Maven JDK	<code>mvn -version</code>	Java version: 21, 非 17/11
JAVA_HOME	<code>\$env:JAVA_HOME</code> (PowerShell)	指向 JDK 21 根目录
父 POM	根 <code>pom.xml</code>	<code>maven.compiler.release=21</code>
PATH	<code>where java</code>	第一条与 JAVA_HOME 一致

常见踩坑: `release version 21 not supported` = Maven 与 java 指向不同 JDK 版本, 或 JAVA_HOME 仍为 17。

9.3. 自检

答辩与联调前必须先跑自检。

```
powershell -File scripts/verify.ps1
```

9.3.1. verify.ps1 三步

步骤	等价命令	覆盖
1	<code>mvn test -pl jieqi-core,jieqi-server,jieqi-ai</code>	142 项自动化测试
2	<code>mvn compile</code>	5 模块编译
3	<code>mvn package -pl jieqi-app -am -DskipTests</code>	产出 <code>unveil-jieqi.jar</code>

预期: 最后一行 OK: `verify passed`。

失败定位: `test` 阶段查 `surefire-reports`; `package` 阶段执行 `mvn install -pl jieqi-app -am -DskipTests` (见 No.9)。

9.4. 启动服务端

服务端是系统唯一权威节点, 必须最先启动。以下六种启动方式覆盖从「最快启动」到「最易调试」的完整梯度, 按推荐优先级排列。核心原则: 优先使用脚本而非裸命

令，脚本内部已处理 SNAPSHOT 过期、JDK 版本探测、Fat JAR 自动构建等常见踩坑点。

9.4.1. 方式对比（推荐顺序）

方式	命令	场景	备注
A 已实现	<code>powershell -File scripts/dev-server.ps1 8887</code>	开发/答辩推荐	package + java -jar; 避免旧 SNAPSHOT
B	<code>powershell -File scripts/run-app.ps1 server-ws 8887</code>	同上	自动探测 JDK 21
C	<code>java -jar jieqi-app/target/unveil-jieqi.jar server-ws 8887</code>	verify 通过后最快	须先 package
D	<code>mvn exec:java -f jieqi-app/pom.xml -am -Dexec.args="server-ws 8887"</code>	快速调试	部分实现 可能加载过期 SNAPSHOT
E	<code>mvn exec:java -f jieqi-app/pom.xml -am</code> → 菜单 3	探索 1 - 10 模式	含 TCP / 本地 AI
F	<code>docker compose up --build</code>	无 JDK 验收机	仅 WS 8887

9.4.2. 启动成功标志

日志/现象	含义
8887 监听 / WsGameServer 启动	WS 服务就绪
无 Address already in use	端口空闲
客户端 loginResult success=true	UserRegistry 正常

9.4.3. 交互菜单 (jieqi-app Main 1 - 10)

项	功能	说明
3	WS 服务器 8887	答辩默认
4	WS 客户端	match / move / replay
9	AI 经 WS 对弈	双 Bot 连服务器
10	观战 watch	输入 roomId
1/2	TCP 8888 服/客	附录 B 调试
5/6/7/8	本地 AI / 性能 / 规则测试	不经网络

9.5. 启动客户端

9.5.1. 控制台 WS 客户端

```
mvn exec:java -f jieqi-app/pom.xml -am -Dexec.args="client-ws ws://127.0.0.1:8887  
player1 123456"
```

推荐：服务端用 run-app.ps1 时，客户端同样用 run-app.ps1 client-ws ... 保持 JAR 版本一致。

常用命令：match → ready → first true → move b 0 b 3 → chat → draw / resign
→ replay (n/p/g) → rematch。

9.5.2. Web 前端

```
cd jieqi-web  
npm install  
npm run dev  
# 浏览器 http://localhost:5173  
# 登录页填 ws://127.0.0.1:8887
```

9.5.3. AI 经 WS (验收补充)

```
mvn exec:java -f jieqi-app/pom.xml -am -Dexec.args="ai-ws ws://127.0.0.1:8887  
ai_bot_1 pw123"
```

9.6. 完整演示流程

三终端手动：

```
终端 1: powershell -File scripts/dev-server.ps1 8887  
终端 2: powershell -File scripts/run-app.ps1 client-ws ws://127.0.0.1:8887 player1  
123456  
终端 3: powershell -File scripts/run-app.ps1 client-ws ws://127.0.0.1:8887 player2  
123456  
流程：match → ready → first → move → 终局 → replay → rematch
```

Web 双标签：终端 1 起 server + 终端 2 npm run dev → 两浏览器各 login → 真人对战 → 终局 → 查看棋局。

9.7. 一键演示

demo.ps1 将前述手动三步封装为一条命令，自动打开三个 PowerShell 窗口并依次启动服务端与两个客户端，适合答辩现场快速进入演示状态。脚本内部通过 Start-Sleep 控制启动时序，避免客户端在服务端就绪前连接失败。

```
powershell -File scripts/demo.ps1
```

窗口	内容	时机
1	server-ws 8887	立即
2	client-ws ... player1	Sleep 3s 后
3	client-ws ... player2	紧接窗口 2

9.8. Docker（实验性）

Docker 部署为补充方案，目标场景是验收机未安装 JDK 21 或 Maven 的情况。当前状态为实验性：仅映射 WS 8887 端口，TCP 8888 与 Web 前端不在容器内，records 未挂载卷导致容器销毁后棋谱丢失。以下为当前可用范围与已知限制。

9.8.1. 前置与启动

要求	检查
Docker 24+ / Compose v2	<code>docker compose version</code>
8887 空闲	<code>netstat -ano findstr 8887</code>

```
docker compose up --build          # 前台
docker compose up --build -d       # 后台
docker compose logs -f jieqi-server
docker compose down
```

9.8.2. Dockerfile 两阶段

阶段	镜像	动作
build	maven:3.9.9-eclipse-temurin-21	<code>mvn package -pl jieqi-app -am -DskipTests</code>
run	eclipse-temurin:21-jre	<code>java -jar unveil-jieqi.jar server-ws 8887</code>

9.8.3. 端口与限制

协议	容器	宿主机	说明
WS JSON	8887	8887 已映射	客户端连 <code>ws://localhost:8887</code>
TCP 8888	—	未实现 未映射	本地 <code>server 8888</code> 或改 <code>compose</code>
jieqi-web	—	未实现 不含	宿主机 <code>npm run dev</code>
records/	—	未实现 未挂卷	容器销毁后棋谱丢失；生产需 <code>volume</code>

9.8.4. Docker vs 本地

维度	Docker	本地 <code>java/mvn</code>
依赖	仅 Docker	JDK 21 + Maven
协议	WS 8887	WS + TCP + 全菜单
客户端	宿主机连 <code>localhost:8887</code>	控制台 + Web
适用	无 Java 快速演示	开发 / AI / 完整验收

9.9. 脚本工具箱

脚本	作用	场景
<code>verify.ps1</code>	<code>test</code> → <code>compile</code> → <code>package</code>	答辩前自检
<code>demo.ps1</code>	三窗口 服+双客	8 分钟演示
<code>dev-server.ps1</code>	<code>package</code> + <code>server-ws</code>	开发推荐
<code>run-app.ps1</code>	<code>package</code> + 任意 CLI 参数	<code>client-ws</code> / <code>server 8888</code>
<code>compile-docs.ps1</code>	Typst → PDF	文档交付
<code>count-loc.ps1</code>	LOC 统计	报告数字

9.10. 常见问题速查

与 `TROUBLESHOOTING.typ` 15 项对齐。

No.	现象	原因	处理
1	Address already in use	8887 被占用	<code>netstat -ano findstr 8887</code> → <code>taskkill</code>
2	Connection refused	未启动 / URL 错	<code>ws://127.0.0.1:8887</code> ；先 <code>server</code> 后 <code>client</code>

No.	现象	原因	处理
3	release version 21 not supported	JAVA_HOME 旧版	JDK 21 + 统一 PATH
4	Unsupported class file major version	低版本 Java 运行	java -version = 21
5	verify.ps1 失败	测试/编译失败	mvn test + surefire-reports
6	AI 走子超时	搜索超预算	改 Easy; 验收用 easy
7	棋盘不同步	未应用 moveResult	以服务器广播为准
8	非法走法未拒绝	本地模式 8 非 WS	WS 客户端验证 valid=false
9	exec:java 行为异常	旧 SNAPSHOT	mvn install -pl jieqi-app -am -DskipTests 或 dev-server.ps1
10	Docker 连不上	端口/地址	docker compose ps; localhost:8887
11	replay 无响应	未终局 / 旧服务端	gameOver 后; 查 .replay.json; 重编译 server
12	匹配后不开局	未 ready / 未 first	双方 ready + first
13	Maven 极慢	网络	settings.xml 国内镜像
14	demo.ps1 闪退	Maven/路径	根目录手动 mvn compile
15	组间消息不一致	混协议 / 大小写	INTERFACE v3.0; 不混 WS+TCP

9.10.1. 诊断命令

目的	命令
全量测试	mvn test
单模块	mvn test -pl jieqi-core / -pl jieqi-ai / -pl jieqi-server
重装 JAR	mvn install -pl jieqi-app -am -DskipTests
查端口	netstat -ano findstr 8887
Docker 日志	docker compose logs -f jieqi-server

详见 TROUBLESHOOTING.pdf。

9.11. 启动成功标志

下表列出各启动方式的预期成功输出，供答辩验收时快速核对。

启动方式	命令	成功标志
全量测试	<code>mvn test</code>	终端输出 <code>BUILD SUCCESS;</code> 末行为 <code>142 tests completed</code>
Fat JAR 打包	<code>mvn package -pl jieqi-app -am -DskipTests</code>	<code>BUILD SUCCESS + target/jieqi-app-*-jar-with-dependencies.jar</code> 存在
WS 服务端	<code>java -jar jieqi-app/target/unveil-jieqi.jar server-ws 8887</code>	输出 <code>WsGameServer started on port 8887</code>
Web 前端	<code>cd jieqi-web && npm install && npm run dev</code>	Vite 输出 Local: <code>http://localhost:5173;</code> 浏览器大厅显示“已连接”（底部状态栏）
自检	<code>powershell -File scripts/verify.ps1</code>	输出 <code>OK: verify passed</code> 或类似成功摘要
演示	<code>powershell -File scripts/demo.ps1</code>	三窗口自动弹出 (server + client1 + client2); client1 输出 login/match/ready 等交互日志
文档编译	<code>powershell -File scripts/compile-docs.ps1</code>	无 typst 报错; docs/ 下 33 份 PDF 全部更新
Docker	<code>docker compose up --build</code>	jieqi-server 容器输出 <code>WsGameServer started on port 8887</code>

10. 产品与用户

10.1. 产品定位

Unveil 的产品定位不是传统意义上的商业棋牌游戏，而是面向课程验收与工程实践展示的揭棋对弈系统。系统以“规则可信、网络可联、AI 可战、棋局可追溯、工程可复

现”为核心目标，在实现基本对弈功能的基础上，进一步补充了 WebSocket 联机、三档 AI、棋谱记录、逐步复盘、Docker 部署与 Typst 文档体系，使其从单一棋类程序扩展为一个完整的课程级对弈产品。

维度	产品定位
产品类型	课程级揭棋对弈系统
核心场景	真人对战、人机对战、AI 自动对弈观战、复盘验收
核心价值	规则复杂度高、AI 有层次、网络协议清晰、工程交付完整
目标用户	课程验收教师、小组成员、测试玩家、AI/规则实现评审者
非目标方向	不追求商业化运营、排行榜生态、付费道具、社交平台化

与成熟商业象棋平台相比，Unveil 并不追求用户规模、排行榜运营、付费体系或精美美术资源，而是突出揭棋规则实现、非完全信息 AI、协议联调、测试验证和工程交付。与普通课程棋类项目相比，Unveil 的复杂度主要体现在揭棋暗子机制、服务端权威规则校验、网络同步、三档 AI 和复盘快照时间线。

产品决策优先级：规则稳定 > AI 可解释 > 复盘可追溯 > 一键运行 > 演示流程清晰 > 商业级 GUI 炫技。架构通过分层设计表与流程图呈现，按 INTERFACE.typ v3.1 通过 8887 端口互联即可完成联调。



Figure 17: 用户旅程 — 四模式入口 • 10 阶段主路径 • 三条典型路径差异

10.2. 用户角色与需求

系统面向六类典型用户，各有不同诉求：

用户角色	主要诉求	对应功能
新手玩家	快速进入对局，理解揭棋规则	随机昵称、emoji 头像、暗子 ? 标记、走子提示
普通玩家	能与真人或 AI 完成完整对局	真人匹配、人机对弈、创建房间、认输、提和
AI 体验者	体验不同强度 AI 的棋力差异	Easy / Medium / Hard 三档 AI、AI 自动对弈观战
规则测试者	验证复杂规则是否正确判罚	禁止送将、长将长捉、40 步无吃子、终局原因展示
课程验收者	快速看懂项目完整度与工程水准	功能矩阵、测试报告、接口文档、演示脚本
开发维护者	能定位问题、复现运行	Maven 多模块、自检脚本、Docker、Typst 文档

10.3. 用户旅程

系统提供三条主要使用路径：真人对战、人机对战和 AI 自动对弈观战。以下从阶段视角展示 Web 端完整旅程。

阶段	用户操作	用户目标	系统响应
1. 登录	输入昵称或使用随机昵称，设置密码	快速获得身份进入系统	创建用户会话，分配 emoji 头像
2. 选择模式	选择真人匹配、人机对弈、AI 观战或创建房间	进入符合需求的对局	建立房间、启动匹配或绑定 AI
3. 选择难度	人机模式选择 Easy / Medium / Hard	获得不同难度的 AI 体验	绑定对应 AiBot 策略
4. 对弈	在 10×9 棋盘点击棋子与目标格走子	完成合法走子	服务端校验后广播 <code>moveResult</code>
5. 信息	接收对手走子、聊天、计时变化	保持双方状态一致	WebSocket 推送棋盘变化与消息

阶段	用户操作	用户目标	系统响应
同步			
6. 辅助操作	快捷消息、提和、认输、暂停、手动加时	完成对局中的辅助决策	服务器广播操作结果
7. 终局	查看胜负、原因、被吃棋子揭晓	明确对局结果	弹窗显示胜负与终局原因
8. 复盘	点击「查看棋局」，逐步查看棋盘快照	回看关键走法与局面变化	请求 <code>replayFrame</code> 并渲染历史局面
9. 重赛	点击「再来一局」，双方确认	快速开启下一局	双方确认后重置房间和棋盘
10. 离开	返回大厅或退出	结束当前会话	清理状态或保留历史棋谱

10.3.1. 三条典型路径差异

步骤	真人对战	人机对战	AI 自动对弈
匹配	需第二名玩家 <code>startMatch</code> 或房间号	单人 <code>startAiGame</code> ， 无需对手	无需玩家，双 Bot 自动对战
准备	双方 <code>Ready</code> + 协商 先手	自动开局	自动开局
走子	双方轮流，65s 步时 限制	用户走 → AI 思考 后应招	双 AI 自动循环，用 户观战
辅助功能	聊天、提和、认输完 整支持	提和/认输可用；聊 天可选	可暂停/结束；无聊 天需求
复盘	终局后 <code>replayFrame</code> 逐步回看	同左	同左（若走完终局）

10.3.2. 控制台旅程（验收补充）

教师或开发者可用 `jieqi-client` 走同一协议链路：`login` → `match` → `ready` → `first` → `move e6 e5` → `replay` → `rematch`。ASCII 棋盘用 `?` 标记暗子，命令行输出 `moveResult.valid` 与 `flipResult`，便于对照 `INTERFACE.pdf` 字段逐项验收。

10.4. 用户痛点与产品价值

痛点	传统做法的问题	Unveil 解决方案
揭棋规则复杂	暗子/翻子/强化士象/长将长捉，人工判断易错漏	服务端 RuleValidator 权威校验 + 89 项 core 单测覆盖全部规则分支
棋盘状态不同步	自写 Socket 通信易出现状态漂移、双方看到不同局面	WebSocket JSON 广播 moveResult，服务器为唯一真相源（Single Source of Truth）
AI 透视作弊	简单 AI 实现可能直接读取对手暗子真实 type，破坏信息差	createAiPublicView 脱敏视角 + AiFairnessTest 不透视约束 + 剩余子力池合法采样
复盘无法还原暗子	走法文本重放无法复现随机翻子结果，复盘局面与实际对局不一致	ReplayTimeline 逐步棋盘快照 + .replay.json 持久化，每帧含完整 board[]
验收不可复现	环境杂乱、依赖手工配置、无可重复的标准验收流程	Maven 多模块 + verify.ps1 自检脚本 + demo.ps1 一键演示 + Docker 容器化
组间无法互联	各组协议字段命名与 messageType 不统一，互操作失败	INTERFACE.typ v3.1 权威规范 + 8887 公共端口 + 全 messageType 表可查

10.5. 核心体验设计

10.5.1. 轻量身份设计

系统不采用复杂注册流程，而是提供昵称、密码与 emoji 头像的轻量身份机制。用户可以输入自定义昵称，也可以使用系统生成的趣味昵称，例如「隐形小炮兵」。这种设计既满足 WebSocket 对局中区分玩家身份的需要，又避免课程演示时在账号注册环节消耗过多时间。

10.5.2. 模式分层设计

系统将对局入口分为真人对战、人机对弈、AI 自动对弈观战与创建房间四类。真人对战用于展示网络同步能力与完整协议消息；人机对弈用于展示 AI 算法能力与难度分层；AI 自动对弈观战用于演示系统稳定运行与双 Bot 搜索决策差异；创建房间则满足指定双方联调与测试需要。

10.5.3. 对弈交互设计

棋盘采用 10×9 坐标布局，暗子以问号标记，明子显示真实棋子名称。用户通过点击棋子和目标格完成走子，前端进行基础交互提示，最终合法性由服务端规则引擎统一判断。该设计避免了客户端和服务端规则不一致的问题，保证网络对局中的状态权威性。

10.5.4. 终局与复盘设计

终局后系统通过弹窗展示胜负结果、终局原因以及被吃棋子揭晓信息（上帝视角），使玩家能够理解对局结束的直接原因。复盘功能不采用单纯走法文本重放，而是基于每一步后的棋盘快照进行回放，能够稳定处理揭棋中的暗子、翻子与随机身份问题，提升对局可追溯性。

10.6. 竞品选择依据

为了评价 Unveil 的产品完整度，本文选取三类参照对象：

类别	代表与说明
商业在线象棋平台	天天象棋、JJ 象棋等，代表成熟在线对战产品，偏大众娱乐与竞技
通用中国象棋 App	以「中国象棋」为名称或关键词的移动端棋类应用，代表轻量单机/联网象棋体验
普通课程棋类项目	Java 五子棋、黑白棋、标准象棋课设，代表同类课程作业的常见实现水平

需要说明的是，Unveil 并不试图在用户规模、商业运营、赛事体系和视觉美术上与成熟商业产品竞争，而是在揭棋规则、非完全信息 AI、协议联调、复盘快照和工程交付方面突出课程项目的技术完整度。

10.7. 竞品对比

维度	Unveil	天天象棋 / JJ 象棋类商业平台	通用中国象棋 App	普通 Java 棋类课设
棋种规则	揭棋变体：暗子、翻子随机、明士明象强化、长将长捉	以标准中国象棋为主，部分平台可能支持变体	多以标准中国象棋为主	多为五子棋、黑白棋或标准象棋
信息结构	非完全信息，对手暗子未知	多为完全信息棋局	多为完全信息棋局	多为完全信息棋局

维度	Unveil	天天象棋 / JJ 象棋类商业平台	通用中国象棋 App	普通 Java 棋类课设
AI 设计	Easy/Medium/Hard 三档，Alpha-Beta + Belief Sampling 可公开说明	偏产品化 AI 或残局训练，算法细节通常不公开	通常提供人机难度，但算法不可见	常见为随机、贪心或简单 Minimax
网络对战	WebSocket JSON 协议，服务端权威校验，课程公共接口互操作	成熟联网匹配与好友对战体系	部分支持联网，部分偏单机	多数无网络或仅本地双人
复盘方式	ReplayFrame 逐步棋盘快照，稳定还原暗子翻子	常见为棋谱/回放/分析功能	常见为棋谱或简单悔棋	多数无复盘或仅文本记录
工程结构	Maven 多模块：core/server/client/ai/app/web	商业闭源工程，外部不可见	多为闭源 App	常见单模块或少量类
文档交付	Typst/PDF 全生命周期文档：接口、测试、部署、产品分析	面向用户帮助页面，非代码级文档	面向用户使用说明	通常只有 README 或课程报告
可测试性	142 项自动化测试 + 自检脚本 + 功能矩阵	外部不可验证内部测试	外部不可验证内部测试	测试覆盖通常较弱
课程适配	高：强调规则、协议、AI、文档、部	低：偏商业应用，不可作为课程参考	中：偏用户体验	中：偏基础编程能力

维度	Unveil	天天象棋 / JJ 象棋类商业平台	通用中国象棋 App	普通 Java 棋类课设
	署，全套可演示			

定位结论：Unveil 的目标不是在 UI 上超越商业象棋产品，而是在课程评分维度（规则正确性、协议互操作、AI 算法深度、测试与文档完整性、工程交付可复现性）上提供完整、可审计、可 8 分钟演示的解决方案。商业平台的价值在用户规模与运营体系，课程项目的价值在技术完整度与可解释性——两者评估维度不同，不存在直接竞争关系。

10.8. 差异化价值

Unveil 的差异化价值主要体现在五个方面：

价值点	具体体现
1. 揭棋规则复杂度高	暗子、翻子随机、士象强化、长将长捉等规则比标准象棋更复杂；服务端 89 项单测覆盖全部规则分支
2. 非完全信息 AI 更有区分度	Hard AI 通过 Belief Sampling 处理对手暗子不确定性，不直接透视；Medium 使用 Agent 编排 Alpha-Beta；三档棋力可感知
3. 服务端权威校验更适合网络对弈	客户端只负责交互，规则由服务端统一判断；createAiPublicView 脱敏视角防止 AI 作弊
4. 复盘采用棋盘快照时间线	不是简单文本重放；每步后棋盘完整快照，能稳定还原暗子翻子后的局面，适配揭棋的随机信息特征
5. 工程交付完整	Maven 5 模块 + Fat JAR + verify.ps1 自检 + demo.ps1 演示 + Docker 部署 + Typst 文档体系，形成代码、测试、部署、接口、文档一体化闭环

10.9. 功能完成度总览

10.9.1. 总览统计

状态	数量	含义
已实现 已实现	48 项	功能已完成，可在主流程中流畅演示
部分实现 待强化	12 项	已有基础实现，但边界测试或体验仍需完善
已实现 实验性	1 项	已提供工程入口（Docker），但不作为主验收承诺
未实现 规划中	4 项	已形成设计方案，预留扩展接口，后续版本实现
合计	65 项	覆盖规则、网络、AI、复盘、工程和文档全部维度

10.9.2. 按模块统计

模块	已实现	待强化	实验性	规划中	说明
规则引擎	12	3	0	0	覆盖七种走法、暗子约束、终局判定、长将长捉
网络对弈	8	2	0	1	WebSocket 主协议、房间、匹配、广播、断线重连
AI 算法	7	3	0	1	三档 AI、搜索优化、Belief Sampling、残局加深
棋谱复盘	5	2	0	1	文字棋谱、复盘时间线、replay.json 持久化
客户端体验	6	2	0	1	棋盘渲染、终局弹窗、复盘控件、聊天表情
工程部署	5	0	1	0	Maven 多模块、Fat JAR、自检脚本、Docker
文档交付	5	0	0	0	Typst/PDF 文档体系，覆盖全生命周期
合计	48	12	1	4	共 65 项

10.10. 产品不足与后续规划

不足	当前影响	后续规划
图形界面偏课程演示	视觉效果和动画细节不如成熟商业平台	增加主题皮肤、走法动画、音效反馈
复盘以快照为主	暂不支持 AI 自动讲解和关键步分析	增加关键步标注、局势评分曲线与 AI 点评
AI 未引入训练模型	棋力弱于成熟商业引擎（依赖手工权重）	引入开局库、残局库与自我对弈数据
用户体系较轻量	暂不支持长期战绩、等级分和排行榜	增加用户战绩统计、ELO 等级分与对局历史
移动端适配有限	手机浏览器体验不足	响应式布局优化，提升移动端可用性

11. 项目管理



Figure 18: 项目管理 — 四人分工 · 四大任务书 63 项 · 交付验收闭环

11.1. 团队分工

成员	学号	主要负责
张恒基 (组长)	2024211301 / 2024210926	架构设计与 AI 算法：主导 Maven 5 模块划分与单向依赖设计；设计 WebSocket JSON 协议 v3.1 (含全部 messageType 枚举与错误码体系)。编写三档 Bot，实现 Alpha-Beta 搜索七项优化 (迭代加深 ID + TT 置换表 2^{20} 槽 Zobrist 索引 + Aspiration Window ± 80 + PVS + LMR + Killer/History 启发式 + Quiescence Search + SEE 过滤亏交换)。设计 Belief Sampling 非完全信息搜索：BoardSampler 均匀采样 + 每次 4 确定化 AB + 期望取优；多 Agent 编排 (ProbabilityAgent/EndgameAgent/SearchAgent)；七维线性加权评估 EnhancedEvaluator；isRepeatedCheckRisk 长将规避 (≥ 5 次扣分 -100000)。 规则引擎与终局判定：设计 Board 聚合根 + ChessPiece 三字段状态模型 (type/virtualType/revealed) + RuleValidator 双层校验 (isValidMove 几何 + isMoveLegal 送将检查) + generateStrictLegalMoves (generate-and-test 范式) + EndgameJudge 固定优先级判定链 (吃将→将死→困毙→80 步和→长将/长捉/兵卒长捉)。positionKey 字符串哈希用于重复计数，ZobristHash 64-bit XOR 用于 AI 置换表索引。 棋谱、复盘与协议：设计 ReplayTimeline 快照时间线 (stepIndex=0 开局帧 + 每步 recordAfterMove 独立 Board 拷贝) + ReplayRecordStore JSON 落盘 (records/{gameId}.replay.json) + WS 协议扩展 (replayRequest/Frame、rematchRequest/Offer/Accept、addTime、pauseGame/resumeGame、startAiBattle、watch)。编写 INTERFACE.typ v3.1 权威规范、JsonMessages 工厂 + JsonMessageTypes 枚举 + BoardJsonMapper 序列化。

成员	学号	主要负责
		文档与项目管理：八类 34 份 Typst 文档架构设计 + template.typ 模板 + FINAL_REPORT.typ 整合撰写。 compile-docs.ps1 编译链 + count-loc.ps1 统计。Git Conventional Commits 规范 (feat/fix/docs/refactor/test)，分支策略 (feat 分支 -> main)，63 项任务拆分。 关键决策：Typst 优先于 LaTeX、WebSocket JSON 优先于 TCP 文本帧、generate-and-test 优先于预计算走法表。
秦博宇	2024211302 / 2024210940	系统架构可视化：绘制 UML 类图 (Board、ChessPiece、RuleValidator、EndgameJudge、ReplayTimeline、AiBot 等关键类)、模块依赖关系图 (core → server/client/ai → app)、对局主流程时序图 (登录、匹配、准备、先手协商、走子、终局)、通信层设计模式图 (外观/单例/工厂/策略/命令)，为团队统一架构认知与答辩演示提供可视化基础； 人机对战质量保障：主导 AI 模块与 Web 前端联调验证，逐项核对六大目标的实现情况，确保所有必选功能已落地且无遗漏；通过 AiFairnessTest 对三档 AI 分别构造随机局面，验证所有输出走法均通过 RuleValidator.isMoveLegal 且 Hard 档不透视对手暗子 (createAiPublicView 脱敏)；利用 BoardUndoTest 在搜索迭代中校验 makeMove/undoMove 的棋盘一致性；在 Web 前端手工模拟难度选择、AI 思考、走子同步、超时处理、提和认输、吃子显示及终局揭晓的完整用户路径；运行 startAiBattle 双 AI 自动对弈并确保所有相关测试集成至 verify.ps1，最终达成 142 项全通过； 服务端开发：参与 WsGameServer 房间管理、匹配队列 (MatchmakingService)、GameRecord 持久化与集成测试。
陈艺博	2024211302 / 2024210931	前端实现：使用 Vue 3、TypeScript、Vite、Pinia 搭建 jieqi-web 工程，完成登录页 (随机昵称 + 预设头像)、大厅页 (真人对战/人机对战/AI 对弈/房间对战四入口)、对局页

成员	学号	主要负责
		(ChessBoard Canvas 10×9 棋盘渲染、暗子 ? 显示、选中高亮、可走点提示)、被吃棋子展示区、局内聊天面板、人机难度选择与 AI 对弈入口等功能模块； WebSocket 协议联调：对接后端登录、匹配、房间、走子、终局、超时、聊天等全部 messageType，确保前端状态与服务器广播同步； Bug 修复：修复棋盘坐标映射错误、Canvas 事件冒泡冲突、线上 WebSocket 地址配置、AI 倒计时不同步、悔棋引发的状态残留等问题； 部署维护：通过服务器 git pull 更新代码并重启前后端服务，保证项目可在线访问。
陈雨飞	2024211005	棋谱与复盘：参与 ReplayTimeline 数据模型设计，编写 ReplayFrame 防御性拷贝与帧编号逻辑的单测（时间线递增、拷贝隔离验证）；在 Web 前端实现复盘播放器 UI —— 步进控件（上一步/下一步/跳至开局/跳至终局）、replayBoard 渲染切换、3s 超时兜底提示、对局中脱敏/终局上帝视角（capturedReveal）切换；局内聊天：设计并实现 Web 端聊天面板 —— 快捷消息模板、3×5 共 15 种预设表情面板、自定义文本输入（≤ 120 字符校验）、消息归属显示（己方/对方 + 时间戳）、新消息提示音开关（chatSoundOn）；对控制台客户端 chat 命令进行同步适配；测试用例编写：编写 DarkPieceRuleTest（暗子走法边界 + 强化士象规则）、RuleEdgeCaseTest 部分子项（送将拦截、照面检测）；配合秦博宇完成 Web 端手工验收路径的回归测试；参与 verify.ps1 自检脚本的测试结果核对与文档同步。

11.2. Git 提交规范

格式：type: 简要描述。type 仅限 feat / fix / docs / refactor / test 五种。

11.3. 四大任务完成状态

四大任务书（`suanfatasks.typ` / `fupantasks.typ` / `chanpintasks.typ` / `wendangtasks.typ`）是组内迭代开发的主线看板：每条任务对应可独立验证的代码单元与验收标准，完成度以「代码落地 + 单测/集成测通过 + 文档同步」三者同时满足为准。

任务文档	范围摘要	子项	进度
suanfatasks	jieqi-ai 三档 Bot + 搜索内核优化 + Agent 编排	18 项	已实现
fupantasks	ReplayTimeline 内存复盘 + JSON 落盘 + WS 协议 + 双端播放器	14 项	已实现
chanpintasks	对局闭环：终局摘要 / 揭晓 / rematch / 复盘 / 演示脚本	16 项	已实现
wendangtasks	00 - 07 文档体系 + Typst/PDF 编译链 + 统计脚本	15 项	已实现
合计	—	63 项	已实现

详细逐项清单（含编号、代码位置、验收证据）见独立任务书文件 `suanfatasks.typ` / `fupantasks.typ` / `chanpintasks.typ` / `wendangtasks.typ`。此处仅保留摘要表。

任务书	子项	进度	核心交付	主要代码位置
suanfatask	8	已实现	三档 Bot + AB 7 项优化 + Belief Sampling + Agent 编排	jieqi-ai/ • AiConfig.java • OptimizedAlphaBeta.java
fupantask	14	已实现	ReplayTimeline 时间线 + JSON 落盘 + WS 协议 + 双端播放器	ReplayTimeline.java • ReplayRecordStore.java • stores/game.ts
chanpintask	6	已实现	终局摘要 + 暗子揭晓 + rematch + demo.ps1 + verify.ps1	GameSummary.java • Game.java • GameView.vue
wendangtask	5	已实现	34 份 Typst → 33 PDF + compile-docs.ps1 编译链	docs/ • INTERFACE.typ • scripts/
合计	63	已实现	—	—

11.3.1. 四大任务交叉验收

验收命令	覆盖任务	预期
<code>mvn test</code>	suanfa + fupan + chanpin 核心逻辑	142/142 BUILD SUCCESS
<code>powershell -File scripts/verify.ps1</code>	chanpin 工程化 + 全模块编译	OK: verify passed
<code>powershell -File scripts/compile-docs.ps1</code>	wendang 全量 PDF	33 PDF 产出无报错
<code>powershell -File scripts/demo.ps1</code>	chanpin 闭环演示	三窗口 WS 对弈可跑通
Web npm run dev + 双浏览器	chanpin Web + fupan 播放器	登录→对弈→终局→复盘

遗留项（不计入四大任务完成度，见 FEATURE_MATRIX P2）：走子错误码细分、jieqi-web Playwright E2E、Hard 残局库、Docker 生产级 Web 同容器部署。

12. 演示与答辩

12.1. 演示时间线（6 - 8 分钟）

时间	操作	说词	预期
0:00	运行 verify.ps1	“首先运行自检脚本，编译、测试、打包全通过”	OK: verify passed

时间	操作	说词	预期
0:30	可选展示菜单 1 - 9	“项目提供统一入口”	菜单显示
1:00	启动 WS 服务器 8887	“启动 WebSocket 服务器”	监听中
1:30	玩家一登录 + 玩家二登录	“两位玩家连接认证”	loginResult 成功
2:00	match → Ready → firstHand	“匹配、准备、协商先手”	gameStart + 初始棋盘
3:00	走子演示: move b7 b4	“合法走子 + 翻子效果”	双方同步 + flipResult
3:30	发非法走法	“服务器拒绝非法着法”	moveResult.valid=false
4:00	AI 演示: ai medium	“三档 AI 对手”	AI 5s 内走子
5:00	触发终局（将死/认输）	“终局判定 + 广播”	gameOver + 摘要
5:30	replay 复盘	“回放棋盘快照时间线”	逐步回看 n/p
6:00	总结	“规则、网络、AI、复盘、工程化五位一体”	—

12.2. 关键命令备忘

```

# 自检
powershell -File scripts/verify.ps1

# 服务端
mvn exec:java -f jieqi-app/pom.xml -am -Dexec.args="server-ws 8887"

# 客户端
mvn exec:java -f jieqi-app/pom.xml -am -Dexec.args="client-ws ws://127.0.0.1:8887
player1 123456"

# 对局命令
match → ready → first → move b 0 b 3 → replay

# Web 前端
cd jieqi-web && npm run dev

```

12.3. 风险预案

风险	备选	话术
Maven 下载慢	用预构建 Fat JAR	“构建已完成，直接启动产物”
网络卡住	改本地人机菜单 6	“网络环节展示协议设计”
AI 思考久	切 ai easy	“Easy 毫秒响应，Hard 展示算法”
verify 失败	展示 mvn-test-output.txt	“今早全量测试已通过”

12.4. 答辩高频问答

Q: 为什么分 5 个模块? A: 领域 (core)、网络 (server/client)、算法 (ai)、入口 (app) 分离。core 不依赖任何外部模块，被所有层复用；server 和 client 只处理通信，不改规则；ai 独立演进，算法升级不影响对局稳定性。

Q: 如何保证规则校验正确? A: 89 项 core 自动化测试覆盖七种走法、暗子约束、送将/照面拒止、将死/困毙全链路。校验在服务端执行，客户端不走终局逻辑。

Q: 暗子和明子走法上有何不同? A: 暗子按 virtualType (原位角色) 走子，明子按真实 type 走子。暗士限九宫、暗象不过河；翻开后明士可出九宫、明象可过河（强化规则）。

Q: AI 是否透视对手暗子? A: 不透视。AI 调用 createAiPublicView 将对手暗子 type 置 UNKNOWN。Hard 档用 Belief Sampling 对对手暗子身份多次采样后求期望。

Q: 三档 AI 的本质区别? A: Easy 不搜索，纯启发+随机；Medium 在公开视角上 Alpha-Beta 搜索；Hard 对外采样的确定化局面上做 Alpha-Beta 后求期望。

Q: 为什么复盘必须保存棋盘快照而非仅走法文本? A: 翻子随机性（暗子真实 type 由服务器随机确定，重走无法复现）+ 信息差（客户端只见公开信息）+ 非确定性（同一棋谱重走结果不一致）。

Q: 一键构建 + 自检怎么实现? A: verify.ps1 依次执行 mvn test → mvn compile → mvn package，三步全过输出 “OK: verify passed”。

13. 总结与展望

Unveil 项目以「规则正确 • 网络互操作 • AI 可解释 • 复盘可追溯 • 工程可自检」为验收主线，历经需求→设计→实现→测试→文档→答辩六阶段，交付一套可运行、可演示、可组间互联的揭棋对弈系统。本章对全项目成果做收束性总结，并诚实列出已知限制与分版本演进路线。

13.1. 项目成果总览

维度	交付物	量化指标
代码	5 Maven 模块 + jieqi-web	63 主代码文件 • 8 500 LOC • 142 项单测全绿
协议	INTERFACE.typ v3.0	8887 WS 公共消息 + 附录 B TCP + 10+ 本组扩展
AI	三档 Bot + Agent 编排	Easy $\leq$ 500ms • Medium/Hard $\approx$ 5s • 16 项 ai 单测
产品	控制台 + Web 双端	登录→对弈→终局→复盘→重赛 全流程
工程	脚本 + Docker + Fat JAR	verify.ps1 • demo.ps1 • dev-server.ps1
文档	00 - 07 八类 + FINAL 整合	34 Typst 源 → 33 PDF + 18 Web 截图 + 6 架构图

13.2. 技术亮点总结

以下提炼本项目区别于常规课设的技术深度。

技术点	说明
暗子状态模型	每个棋子同时持有 <code>type</code> （初始化随机分配的真实身份）与 <code>virtualType</code> （原位角色决定走法），辅以 <code>revealed</code> 标志； <code>Board.initBoard()</code> 通过 <code>Collections.shuffle</code> 预分配类型， <code>RandomRevealService</code> 仅防客户端伪造，不重新随机
双层走法校验	<code>RuleValidator.isValidMove</code> 判定几何合法（蹩腿/炮架/暗子 <code>virtualType</code> ）， <code>RuleValidator.isMoveLegal</code> 通过 <code>makeMove</code> → <code>isInCheck</code> → <code>unmakeMove</code> 确保不送将； <code>generateStrictLegalMoves</code> 为 <code>generate-and-test</code> 范式
终局优先级判定	<code>EndgameJudge.checkAfterMove</code> 按固定优先级链判定：吃将 → 将死 → 困毙 → 80 半步无吃子和 → 重复局面（长将/长捉子判定）； <code>positionKey</code> 字符串哈希用于重复计数，每次吃子后清空

技术点	说明
Belief Sampling 非完全信息搜索	Hard AI 通过 BoardSampler 从剩余子力池（2 车 2 马 2 炮 5 兵 2 士 2 象）均匀采样对手暗子身份，对每个 candidate 执行 beliefSamples=4 次确定化 AB 搜索，取期望分最高走法；每采样独立 new OptimizedAlphaBeta() 消除 TT 跨采样污染
Alpha-Beta 优化七件套	迭代加深 + 置换表 TT (2 ²⁰ 槽 Zobrist 索引) + Aspiration Window ±80 + PVS + LMR (quiet 降 depth) + Killer/History 启发式 + 静态搜索 + SEE 过滤亏交换
复盘快照而非走法重放	揭棋暗子随机身份导致单纯走法文本无法重放——同走法序列每次执行翻子结果不同；因此采用 ReplayFrame 快照时间线 (stepIndex=0 开局帧 + 每步后独立 Board 拷贝)，支持对局中脱敏/终局上帝视角
工程自检体系	verify.ps1 (mvn test → compile → package) + demo.ps1 (三窗口自动化) + dev-server.ps1 (防 SNAPSHOT 过期) + compile-docs.ps1 (34→33 PDF)，四条脚本构造可重复验证的验收闭环

13.3. 已完成功能

各模块的详细功能说明与实现细节参见对应设计章节（§4 规则引擎、§5 AI 算法、§6 客户端、§7 棋谱与复盘），产品级功能对比见 §10 产品与用户。此处仅提供答辩速查级总览。

模块	一句话	状态
已实现 规则引擎	七种走法 + 暗子 + 强化士象 + 终局全链路，89 项单测	已实现
已实现 WebSocket 对弈	8887 公共接口：匹配/房间/超时/聊天/提和/认输/加时/暂停	已实现
已实现 TCP 兼容	8888 附录 B 文本帧，legacy 联调	已实现
已实现 三档 AI	Easy 启发 / Medium AB+7 项优化 / Hard Belief，16 项单测	已实现
已实现 棋谱与复盘	棋谱 + 时间线 + JSON + replayRequest/Frame	已实现

模块	一句话	状态
已实现 Maven 多模块	5 模块 + Fat JAR + verify + demo	已实现
已实现 控制台客户端	10 模式 + ConsoleUI + replay n/p/g	已实现
已实现 Web 前端	Vue3 全流程 + 18 张界面截图入 FINAL	部分实现
已实现 文档体系	34 Typst → 33 PDF, 八大类	已实现
已实现 测试	142/142 BUILD SUCCESS	已实现

13.4. 已知限制

诚实列出当前边界——不影响课程主验收，但答辩时应主动说明。

13.4.1. P1 — 短期应强化

限制	现状	影响	缓解
走子错误原因不细分	<code>RuleValidator</code> 返回 <code>boolean</code> ; <code>processMove</code> 仅「非法走法/不能送将」	客户端无法精确提示; 与 <code>errorCode</code> 2001/2002 未对齐	V1.1 引入 <code>RejectionReason</code> 枚举
长捉分类简化	<code>findChaseTarget</code> 用 <code>isValidMove</code> 近似「捉」	隔子捉/连环捉/将杀捉未区分; 极端局面需人工裁	V1.1 补 10+ 边界用例 + 裁判三分
Web 复盘部署依赖	UI 与协议已就绪	旧 SNAPSHOT 服务端无 <code>handleReplayRequest</code> 则超时	<code>dev-server.ps1</code> 重编译部署
双端校验反馈不一致	服务端权威; 客户端预校验弱	无效 WS 往返多	V1.1 客户端映射相同 <code>reason</code>
验收标准 C12 长将长捉	主路径单测通过	FEATURE_MATRIX 仍标“待强化”	补集成测后改“已实现”

13.4.2. P2 — 中期可优化

限制	说明	影响范围
Hard AI Belief 开销	<code>beliefSamples=4</code> , <code>maxCandidatesForBelief=6</code> ;	Hard 棋力上限; 验收可演示算法

限制	说明	影响范围
	每采样独立迭代加深，实际深度由时间预算与局面分支数决定	
残局库缺失	EndgameAgent 仅加深时间，无表库	残局精确度依赖搜索到底
generateAllMoves 暴力	每子枚举 90 格	AI 节点吞吐；Medium 深度上限
局面哈希双轨	Game 用字符串 key；AI 用 Zobrist	重复判定与 TT 未统一
jieqi-client 无单测	依赖 server 集成间接覆盖	客户端回归风险
jieqi-web 无 E2E	无 Playwright 自动化	UI 回归靠手工截图
Docker 不完整	仅 WS 8887；无 Web 同容器；records 未挂卷	生产部署需扩展 compose

13.4.3. P3 — 已知不追求（课设边界）

项	说明
商业级 GUI	不追求 3D/动画/皮肤商城；Web 满足演示
用户增长/付费	无账号体系、无排行榜（V2 再议）
Redis 分布式房间	单机内存 RoomManager 已满足验收
断线重连保局	当前断线倾向判负；非 P0
Null-Move / ISMCTS	AI 任务书明确不做
强化士象争议	按组内协议实现；与传统揭棋可能不一致

13.5. 后续规划

13.5.1. 版本路线图

版本	时间	目标	成功标准
V1.0	当前	课程验收交付	verify 全绿 • 8 分钟演示 • INTERFACE 对齐

版本	时间	目标	成功标准
V1.1	短期 1 - 2 周	规则可观测 + 复盘生产 + 长捉边界	errorCode 2001/2002 • Web 复盘稳定 • 长捉测例 +10
V2.0	中期 1 - 2 月	产品化 + 互操作	旁观大厅 • 联调矩阵 • Web 体验 polish
V3.0	远期	竞技与智能升级	残局库 • NN 评估 • 移动端

13.5.2. V1.1（短期）— 规则与复盘加固

任务	内容	负责模块
R1	RuleValidator → ValidationResult(reason, code)	jieqi-core
R2	WsGameServer error 消息对齐 INTERFACE 错误码	jieqi-server + INTERFACE.typ
R3	Web/控制台 toast 显示具体规则名	jieqi-web / jieqi-client
R4	长捉: isMoveLegal 过滤 + 将/杀/捉分类	EndgameJudge
R5	RuleEdgeCaseTest 补隔子捉/连环捉等 10 用例	jieqi-core test
R6	Web 复盘: dev-server 部署文档 + 集成冒烟清单	docs + scripts
R7	FEATURE_MATRIX 长将/长捉/错误码待强化→已实现	docs

13.5.3. V2.0（中期）— 产品化与互操作

方向	规划内容
旁观大厅	watch 从实验命令升级为 Web 入口; 房间列表 + 只读棋盘
排行榜	Elo/胜率统计; 需持久化用户战绩 (可选 SQLite)
多组联调矩阵	与 2 - 3 组按 INTERFACE v3.0 互操作测试表; 记录兼容差异
Web 完整体验	音效/动画 polish • 移动端响应式布局 • 断线提示优化
走子错误 UX	棋盘高亮违规格 • 建议合法着法列表
Docker 全栈	compose 含 jieqi-web nginx + records volume + 8887/8888

13.5.4. V3.0（远期）— 竞技与智能

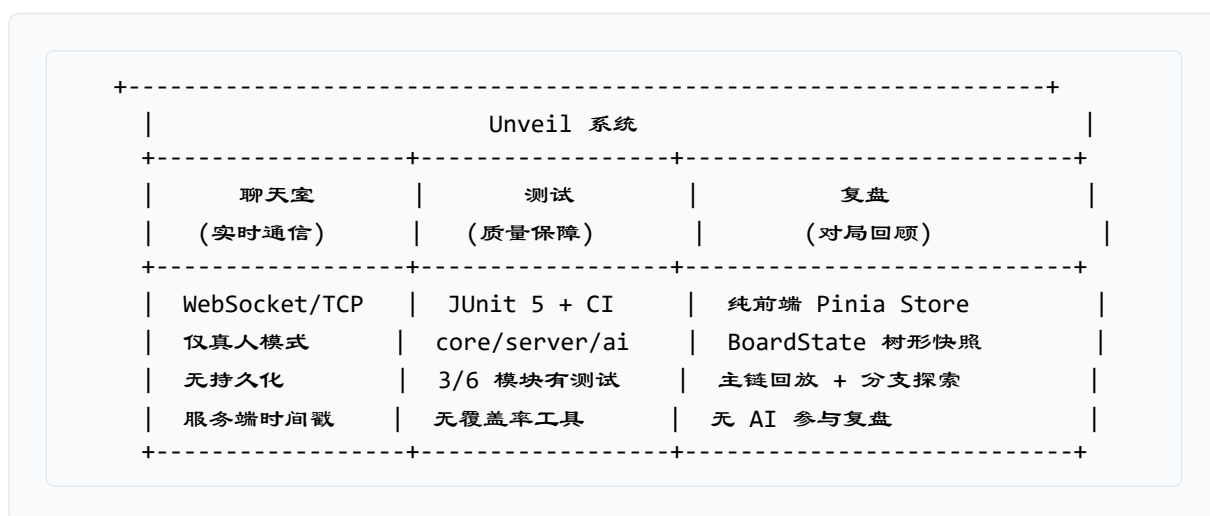
方向	规划内容
残局数据库	车兵/马兵等典型残局表；EndgameAgent 查表 + 搜索 hybrid
神经网络评估	替代/辅助 PST 子力评估；需训练数据与推理框架
移动端	Vue 响应式或 Uni-app；触屏走子手势
分布式	Redis 房间状态 • 多实例 WS 负载均衡（超出课设）
更强 Hard AI	ISMCTS / 更深 Belief • 对手建模非均匀采样

13.5.5. 四大任务与版本对应

任务书	V1.0 状态	V1.1+ 演进
suanfatasks	已实现 三档 Bot + AB 优化	Hard 深度/采样调参 • 走法生成性能
fupantasks	已实现 时间线 + 协议 + 落盘	Web E2E • 对局中脱敏复盘 UI
chanpintasks	已实现 闭环 + demo/verify	错误码 UX • stats/record 命令
wendangtasks	已实现 34 Typst + PDF	V1.1 同步 FEATURE_MATRIX 状态变更

13.6. 关键子系统关联总览

聊天室、测试、复盘三项功能在架构上相互独立，各自职责清晰：



Listing 21: 聊天室、测试、复盘三项功能架构关系

- 聊天室是实时通信层，依赖 WebSocket 广播机制，服务端与前端双重校验仅真人可用

- 测试是横切关注点，覆盖核心领域逻辑（89 项）与协议集成（37 项），CI 自动化保障
- 复盘是前端交互层，以 BoardState 树记录对局快照并支持事后探索，与服务端持久化解耦

13.7. 结语

Unveil 在课程要求的面向对象设计、网络对弈、规则校验、AI 博弈、文档与测试各维度均给出可运行、可审计、可演示的交付物。V1.0 的目标是验收通过而非商业产品完备；V1.1 聚焦「规则可解释、复盘可生产、长捉可辩护」三项答辩高频追问；V2.0 及以后视课程后续或开源维护意愿再迭代。

项目仓库：Unveil/ • 主协议：docs/INTERFACE.typ v3.0 • 自检：powershell -File scripts/verify.ps1 • 演示：powershell -File scripts/demo.ps1

14. 附录

14.1. A. TCP 文本帧扩展协议（摘要）

TCP 端口 8888，帧格式：msgType|payloadByteLength|payload\n

消息类型	msgType	说明
MSG_LOGIN	1	客户端登录
MSG_MOVE	2	走子请求
MSG_GAME_STATE	3	游戏状态同步
MSG_ERROR	4	错误消息
MSG_QUIT	5	退出
MSG_GAME_OVER	6	游戏结束
MSG_BOARD_STATE	7	棋盘状态同步
MSG_DRAW_REQUEST	8	提和
MSG_RESIGN	9	认输
MSG_CHAT	10	聊天

TCP 扩展支持多盘 gameId 字段，完整规范见 INTERFACE.typ 附录 B。

14.2. B. 文档体系完整清单

类别	文档	格式
00-概览	PROJECT_OVERVIEW • FEATURE_MATRIX • GLOSSARY	Typst + PDF + MD
01-需求	REQUIREMENTS • ACCEPTANCE_CRITERIA	Typst + PDF + MD
02-设计	ARCHITECTURE • DOMAIN_MODEL • AI_DESIGN • RULE_ENGINE_DESIGN • REPLAY_DESIGN	Typst + PDF + MD
03-接口	INTERFACE • MESSAGE_EXAMPLES • INTEROP	Typst + PDF + MD
04-部署	BUILD_AND_RUN • DOCKER_DEPLOYMENT • TROUBLESHOOTING	Typst + PDF + MD
05-测试	TEST_PLAN • TEST_CASES • TEST_REPORT • COMPLETION_REPORT	Typst + PDF + MD
06-产品	PRODUCT_REQUIREMENTS • USER_JOURNEY • COMPETITOR_ANALYSIS	Typst + PDF + MD
07-答辩	FINAL_REPORT • DEMO_SCRIPT • DEFENSE_QA	Typst + PDF + MD
根目录	README • TEAM • TASKS_COMPLETION_STATUS	Typst + PDF + MD
任务拆解	suanfatasks • fupantasks • chanpintasks • wendangtasks	Typst + PDF + MD

合计：34 份 Typst 源文件 → 33 份 PDF + `template.typ` 共享模板。

14.3. C. 术语速查

术语	定义
揭棋	中国象棋变体：开局仅将/帅明置，其余 15 子暗置并按原位角色走子；首次移动/吃子后随机翻开
暗子	未翻开的棋子：真实 <code>type=UNKNOWN</code> ，按 <code>virtualType</code> （原位角色）走子
明子	已翻开的棋子：按真实 <code>type</code> 走子；明士可出九宫、明象可过河（强化规则）
<code>virtualType</code>	棋子所在格开局原位对应的象棋角色类型，暗子走子依据
强化士象	揭棋特有规则：翻开后的士可出九宫斜走全场、象可过河走田字；暗士仍限九宫、暗象不过河

术语	定义
翻子	暗子首次移动 <code>revealed = true</code> ，真实 <code>type</code> 在 <code>Board.initBoard()</code> 阶段已通过 <code>Collections.shuffle</code> 随机分配并写入棋盘； <code>RandomRevealService</code> 仅清除客户端伪造 <code>type</code> ，走子后写回服务端真实类型
复盘时间线	<code>ReplayTimeline: List<ReplayFrame></code> ，对局每步（含开局）的棋盘完整快照
Belief Sampling	Hard AI 算法：对对手暗子身份多次随机采样，在每个确定化局面上 AB 搜索后取期望收益最高的走法
ZobristHash	64 位局面哈希，用于置换表索引和长将/长捉重复局面判定
<code>capturedReveal</code>	终局 <code>gameOver</code> 消息字段：对局中所有被吃暗子的真实身份揭晓（上帝视角）

14.4. D. 构建命令速查

```
# 全量测试
mvn test

# 编译
mvn compile

# 打包 Fat JAR
mvn package -pl jieqi-app -am -DskipTests

# 启动服务端 (WS 8887)
java -jar jieqi-app/target/unveil-jieqi.jar server-ws 8887

# 自检
powershell -File scripts/verify.ps1

# 演示
powershell -File scripts/demo.ps1

# 编译全部文档 PDF
powershell -File scripts/compile-docs.ps1

# 编译协议权威 PDF
typst compile docs/INTERFACE.typ docs/INTERFACE.pdf --root docs

# Web 前端
cd jieqi-web && npm install && npm run dev
```

14.5. E. 版本历史

版本号方案：项目对外交付版本自 V0.1（2026-05-22）起算，V1.0 为课程验收基线。INTERFACE 协议独立维护 v0.0.0 → v3.1 版本线（见 INTERFACE.typ 附录 C）。

版本	日期	主要变更
V0.1	2026-05-22	Maven 多模块骨架 (core / server / client / ai / app / record); Board、ChessPiece、RuleValidator、Game 领域模型与棋规; TCP 文本帧协议 (msgType len payload) 与 GameServer / GameClient; Alpha-Beta 基础搜索、局面评估函数与 Zobrist 哈希; 控制台 ASCII 棋盘与交互菜单 1 - 10
V0.2	2026-05-29	INTERFACE.typ v3.0: WebSocket + JSON 升为正文主协议, 默认端口 8887; TCP 协议迁移至附录 B; WsGameServer / WsGameClient WebSocket 实现; 课程公共 messageType 全量对接 (login → gameOver) 与 errorCode 体系; 组间联调清单与典型时序图
V0.3	2026-06-08	AI 三档 Bot: Easy Top-K 随机 / Medium 迭代加深 AB+TT / Hard Belief 采样; AgentOrchestrator 编排 (Probability → Endgame → Search); GameRecord 棋谱记法、ReplayTimeline 内存时间线、replay.json 落盘; 局内聊天 chatMessage、提和/认输/加时/暂停辅助功能; jieqi-web Vue 3 前端骨架: Login/Lobby/Game 三页与 Canvas 棋盘
V0.4	2026-06-15	Web 前端产品闭环: 随机昵称 + emoji 头像、被吃棋子信息差展示; 终局弹窗 (胜负 + 原因 + 上帝视角 capturedReveal); 聊天快捷语 + 15 种 emoji 面板; 登录/匹配/房间创建二次确认防误触; BoardAiPublicViewTest 暗子脱敏校验、AiFairnessTest 公平性测例
V1.0	2026-06-18	课程验收交付基线; 142 项测试全绿 (core 89 + ai 16 + server 37)、BUILD SUCCESS; 34 份 Typst 文档与 33 份 PDF, 含本 FINAL_REPORT 整合终稿; 18 张 Web 截图 + 6 张架构图入终稿; verify.ps1 自检 / demo.ps1 演示 / dev-server.ps1 开发部署脚本; Fat JAR 单文件启动、Docker Compose WS 8887; INTERFACE.typ v3.1: 27 条自检清单、§ 14 实现状态标注、附录 C 版本历史
V1.1	计划 2026-06-25	RuleValidator 返回 RejectionReason 枚举, errorCode 2001/2002 对齐 INTERFACE; 长将/长捉分类强化 (将/杀/捉三

版本	日期	主要变更
		分、隔子捉/连环捉 10+ 边界用例); Web 复盘生产部署稳定、双端校验反馈一致; FEATURE_MATRIX 长将/长捉/错误码 待强化→已实现
V2.0	计划 1-2 月	旁观大厅 (房间列表 + 只读棋盘 Web 入口); 多组联调矩阵 (2-3 组 INTERFACE v3.1 互操作测试表); Elo/胜率排行榜 (SQLite 持久化); Web 体验 polish (移动端响应式、音效/动画、走子错误 UX); Docker 全栈 compose (jieqi-web nginx + records volume + 8887/8888)
V3.0	远期	残局数据库 (车兵/马兵典型残局表 + EndgameAgent 查表 hybrid); 神经网络局面评估 (替代/辅助手工权重 PST); 移动端 (Vue 响应式或 Uni-app, 触屏走子手势); 更强 Hard AI (ISMCTS / 更深 Belief / 对手建模非均匀采样); 分布式 (Redis 房间状态、多实例 WS 负载均衡)

Unveil 揭棋对弈系统 — 最终报告 (初版)

第一组 • 张恒基 秦博宇 陈艺博 陈雨飞 • 2026-06-18